

1. 여행 배낭을 꾸리며. 이 문서는 타원 곡선을 스스로 익히려고 쓴 답사기다. 목적지는 두 곳이다. 하나는 곡선 위의 점을 세는 *Schoof* 알고리즘—순수 수학이 어떻게 다항 시간 마술을 부리는지 보여 주는 봉우리고, 다른 하나는 그 곡선을 무기로 쓰는 타원 곡선 암호—이산 로그의 어려움 위에 세운 서명 ECDSA 다. 가는 길에 유한체, 다항식과 그 빠른 곱셈(NTT), 나눗셈 다항식, 이산 로그 공격 삼총을 차례로 지낸다.

답사에는 사연이 있다. 전에 같은 것을 *math/big* 만으로 짰더니 Schoof가 너무 느려, 60비트 소수 하나 세는데 밤을 새웠다. 그래서 이번 판은 점 세기 엔진을 통째로 **uint64** 산술 위에 다시 세우고 다항식 곱셈에 수론적 변환(NTT)을 얹었다—학습용 장난감치고는 제법 이빨이 있다. 하지만 이 글의 진짜 목적은 속도가 아니라 이해다. 그래서 함수 하나하나에 “무엇을”과 함께 “왜”를 적으려 했고, 곡선의 역사와 뒷이야기도 곳곳에 끼워 넣었다.

2. 프로그램은 한 Go 패키지 *elliptic* 으로 나온다. `gtangle` 하면 주 출력 `elliptic.go`와 시험 파일 `elliptic_test.go`가 함께 떨어진다. 층은 아래에서 위로 쌓인다.

유한체	uint64 위의 F_p 산술
다항식	$F_p[x]$ 와 NTT 빠른 곱셈
타원 곡선	<i>big.Int</i> 위의 군 법칙 (그림과 함께)
나눗셈 다항식	등분점을 붙드는 ψ_n
Schoof	프로베니우스로 점 세기
이산 로그	Shanks, Pollard ρ , Pohlig–Hellman
ECDSA	secp256k1 위의 서명

낮은 두 층(유한체·다항식)은 속도가 생명이라 힘을 꺼리는 **uint64**로 짜고, 높은 층(곡선·암호)은 256비트 수를 다뤄야 하니 *big.Int*로 짤다. Schoof는 이 두 세계에 다리를 놓아, 큰 곡선의 문제를 작은 체의 빠른 산술로 푼다.

```
package elliptic
import (
    "errors"
    "io"
    "math/big"
    "math/bits"
    "sort"
    "sync"
    "crypto/rand"
)
< 유한체 4 >
< 다항식 10 >
< 타원 곡선 35 >
< 나눗셈 다항식 46 >
< Schoof 알고리즘 53 >
< 이산 로그 문제 76 >
< ECDSA 91 >
```

3. 시험 파일은 패키지 안에서 돌며 내부 함수까지 들여다본다. 각 장이 제 시험을 `elliptic_test.go`에 조금씩 보태므로, 여기서는 임포트만 모아 둔다. 무작위성은 재현 가능하도록 씨앗을 고정한 `math/rand/v2`의 PCG를 쓰되, 암호 연산(키 생성·서명)만은 진짜 `crypto/rand`를 쓴다.

(`elliptic_test.go` 3) ≡

```
package elliptic
import (
    "crypto/rand"
    "crypto/sha256"
    "math/big"
    mrand "math/rand/v2"
    "testing"
)
```

이 코드는 9, 30, 31, 32, 33, 43, 44, 51, 71, 72, 73, 74, 88, 89, 96절에도 정의되어 있다.

4. **유한체 F_p .** 타원 곡선 암호의 무대는 실수 평면이 아니라 유한체다. 소수 p 에 대해 $\{0, 1, \dots, p-1\}$ 에 “ p 로 나눈 나머지” 셈을 얹으면 덧셈·뺄셈·곱셈은 물론 0 아닌 수의 나눗셈까지 되는 체 F_p 가 된다. 시계가 좋은 비유다: 12시에서 세 시간을 더 가면 3시이듯, F_{13} 에서 $12 + 3 = 2$ 다. 다만 시계와 달리 곱셈의 역원까지 있다는 점이 요긴하다— F_{13} 에서 $5 \times 8 = 40 = 1$ 이니 $1/5 = 8$ 이다.

이 장은 $p < 2^{63}$ 인 소수에 대한 F_p 를 **uint64** 하나로 구현한다. 원소는 $[0, p)$ 의 대표 잉여로 저장하고, 힙 할당이 전혀 없다. 굳이 이렇게 하는 데는 쓰라린 사연이 있다. 전에 같은 것을 *math/big*으로 짰더니 Schoof 알고리즘이 소수 하나 세는 데 하세월이었다. *big.Int*는 아무리 작은 수라도 힙에 놓고, 덧셈 한 번에도 포인터를 쫓는다. 반면 **uint64**는 레지스터에서 태어나 레지스터에서 죽는다. 수백만 번 반복되는 안쪽 고리에서 이 차이는 수십 배로 벌어진다. 물론 256비트 소수를 쓰는 ECDSA 층은 *big.Int*로 갈 수밖에 없지만 (그쪽은 연산 횟수가 몇백 번뿐이라 아무래도 좋다), 다항식을 수십만 번 곱하는 점 세기 엔진은 이 작고 빠른 체 위에 세운다.

$p < 2^{63}$ 이라는 제한은 게으름이 아니라 설계다. 두 원소가 2^{63} 미만이면 합이 **uint64**를 넘치지 않아 덧셈에 128비트 중간값이 필요 없다.

〈 유한체 4 〉 ≡

```
type Fp struct {
    p uint64
}

func NewFp(p uint64) *Fp {
    if p < 2 ∨ p >> 63 ≠ 0 {
        panic("elliptic: Fp의 p는 2 ≤ p < 2^63이어야 한다")
    }
    return &Fp{p: p}
}
```

이 코드는 5, 6, 7, 8절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

5. 날개 연산 넷은 손가락 셈이다. 덧셈은 넘치면 p 를 한 번 빼고, 뺄셈은 모자라면 p 를 한 번 끈다. *fromInt*는 음수일 수도 있는 정수를 $[0, p)$ 로 데려온다—Go의 %는 피제수의 부호를 따르므로 음수에는 p 를 한 번 더 얹어야 한다.

〈유한체 4〉 +=

```

func (f *Fp) reduce(a uint64) uint64 { return a % f.p }
func (f *Fp) fromInt(v int64) uint64 {
    r := v % int64(f.p)
    if r < 0 {
        r += int64(f.p)
    }
    return uint64(r)
}
func (f *Fp) add(a, b uint64) uint64 { return addmod(a, b, f.p) }
func (f *Fp) sub(a, b uint64) uint64 { return submod(a, b, f.p) }
func (f *Fp) neg(a uint64) uint64 {
    if a == 0 {
        return 0
    }
    return f.p - a
}

```

6. 진짜 일꾼은 법이 매개변수인 다음 네 함수다. **Fp**의 메서드로 만들지 않고 따로 둔 것은 뒤에 나올 NTT가 자기만의 법 세 개를 들고 이 함수들을 다시 찾아올 것이기 때문이다.

곱셈이 볼거리다. $a, b < p < 2^{64}$ 의 곱은 128비트까지 자라는데, Go에는 128비트 정수가 없다. 대신 *math/bits*가 두 마디를 내주는 곱셈 *Mul64*와 두 마디를 받는 나눗셈 *Div64*를 갖추고 있어, 곱하고 나머지를 취하는 일이 정확히 두 명령이다. *Div64*는 상위 마디가 법 이상이면 몫이 넘친다고 패닉하지만, $a, b < m$ 이면 $ab < m \cdot 2^{64}$ 라 상위 마디가 늘 m 미만이니 안전하다.

〈유한체 4〉 +=

```
func mulmod(a, b, m uint64) uint64 {
    hi, lo := bits.Mul64(a, b)
    _, r := bits.Div64(hi, lo, m)
    return r
}

func addmod(a, b, m uint64) uint64 {
    s := a + b // a, b < m < 263이니 넘치지 않는다
    if s ≥ m {
        s -= m
    }
    return s
}

func submod(a, b, m uint64) uint64 {
    if a ≥ b {
        return a - b
    }
    return m - b + a
}
```

7. 거듭제곱은 지수의 이진 표현을 따라 제공하며 오르는 정석 그대로다. 지수가 $e = e_0 + 2e_1 + 4e_2 + \dots$ 일 때 $a^e = \prod_{e_i=1} a^{2^i}$ 이므로, 비트를 하나씩 밀며 밀을 제공해 간다. $e < 2^{64}$ 라도 예순네 걸음이면 끝난다.

〈유한체 4〉 +=

```
func powmod(a, e, m uint64) uint64 {
    r := uint64(1) % m
    a %= m
    for; e > 0; e >>= 1 {
        if e & 1 == 1 {
            r = mulmod(r, a, m)
        }
        a = mulmod(a, a, m)
    }
    return r
}
```

8. 나눗셈은 페르마의 작은 정리에 기댄다. 소수 p 와 p 의 배수가 아닌 a 에 대해 $a^{p-1} \equiv 1 \pmod{p}$ 이므로 $a^{-1} = a^{p-2}$ 다. 1640년에 페르마가 친구 프레니클에게 보낸 편지에서 “증명을 보내 주고 싶지만 너무 길어질까 두렵다”며 증명 없이 알려 준 그 정리다—여백이 모자라다던 마지막 정리보다야 사정이 낮지만, 증명 미루는 버릇은 한결같다. 확장 유클리드 호제법이 더 빠르지만, 예순 걸음 남짓의 거듭제곱도 충분히 싸고 코드가 곱다.

〈유한체 4〉 +=

```
func (f *Fp) mul(a, b uint64) uint64 { return mulmod(a, b, f.p) }
func (f *Fp) pow(a, e uint64) uint64 { return powmod(a, e, f.p) }
func (f *Fp) inv(a uint64) uint64 { return powmod(a, f.p-2, f.p) }
```

9. 시험은 메르센 소수 $2^{61} - 1$ 위에서 *math/big* 과 대조한다. 답안지 채점을 채점 대상보다 믿음직한 자에게 맡기는 셈이다. 역원은 $a \cdot a^{-1} = 1$ 만 확인하면 된다.

〈elliptic_test.go 3〉 +=

```
const p61 = 2305843009213693951 // 메르센 소수  $2^{61} - 1$ 
var rng = mrand.New(mrand.NewPCG(42, 2026))
func TestFp(t *testing.T) {
    f := NewFp(p61)
    P := new(big.Int).SetUint64(p61)
    for range 200 {
        a, b := rng.Uint64() % p61, rng.Uint64() % p61
        A := new(big.Int).SetUint64(a)
        B := new(big.Int).SetUint64(b)
        check := func(op string, want *big.Int, got uint64) {
            if want.Mod(want, P).Uint64() != got {
                t.Fatalf("%s(%d, %d) = %d 이 되었다", op, a, b, got)
            }
        }
        check("add", new(big.Int).Add(A, B), f.add(a, b))
        check("sub", new(big.Int).Sub(A, B), f.sub(a, b))
        check("mul", new(big.Int).Mul(A, B), f.mul(a, b))
        if a != 0 & f.mul(a, f.inv(a)) != 1 {
            t.Fatalf("inv(%d) 가 역원이 아니다", a)
        }
    }
}
```

10. 다항식. Schoof 알고리즘은 곡선 위의 점이 아니라 다항식 위에서 논다. 프로베니우스 사상이니 ℓ -등분점이니 하는 등장인물이 모두 $F_p[x]$ 의 원소로 분장하고 나오기 때문에, 먼저 다항식 산술을 갖추어야 한다. 이 장의 목표는 사칙과 합동식 산술, 그리고—이 프로그램의 속도를 결정짓는—빠른 곱셈이다.

계수는 낮은 차수부터 `[]uint64` 한 판에 담는다. 첨자가 곧 차수라서 $3x^3 + 2x + 1$ 은 $\{1, 2, 0, 3\}$ 이다. 필기 순서와 반대라 처음에는 어색하지만, i 차 항이 $c[i]$ 에 있다는 규칙은 곱셈에서 $c[i+j]$ 처럼 첨자 셈이 그대로 지수 법칙이 되는 즐거움을 준다. 연산은 언제나 새 다항식을 돌려주고 피연산자를 건드리지 않으며, 최고차 계수가 0이 아니도록(상수항만은 예외로 남겨) 다듬어진 상태를 유지한다. 그래서 Deg 는 길이에서 1을 빼면 끝이고, 영 다항식의 차수는 편의상 0으로 친다.

〈다항식 10〉≡

```
type FpPoly struct {
    f *Fp
    c []uint64
}

func (f *Fp) Poly(coeffs ... uint64) FpPoly {
    c := make([]uint64, len(coeffs))
    for i, v := range coeffs {
        c[i] = f.reduce(v)
    }
    if len(c) == 0 {
        c = []uint64{0}
    }
    return FpPoly{f, c}.trim()
}

func (p FpPoly) Deg() int { return len(p.c) - 1 }
func (p FpPoly) isZero() bool { return len(p.c) == 1 & p.c[0] == 0 }
```

이 코드는 11, 12, 13, 14, 15, 16, 18, 19, 21, 22, 24, 26, 27, 28, 29절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

11. 덧셈·뺄셈 뒤에는 최고차 계수가 상쇄되어 0이 될 수 있으니, *trim* 이 웃자란 0들을 쳐낸다. 상수항까지 치지 않는 것은 영 다항식도 길이 1은 가져야 하기 때문이다.

〈다항식 10〉 +=

```
func (p FpPoly) trim() FpPoly {
    n := len(p.c)
    for n > 1 ∧ p.c[n-1] ≡ 0 {
        n--
    }
    return FpPoly{p.f, p.c[:n]}
}

func (p FpPoly) clone() FpPoly {
    return FpPoly{p.f, append([], uint64(Λ), p.c...)}
}

func (p FpPoly) equal(q FpPoly) bool {
    if len(p.c) ≠ len(q.c) {
        return false
    }
    for i := range p.c {
        if p.c[i] ≠ q.c[i] {
            return false
        }
    }
    return true
}
```

12. 덧셈은 짧은 쪽을 긴 쪽에 포깁니다.

〈다항식 10〉 +=

```
func (p FpPoly) Add(q FpPoly) FpPoly {
    f := p.f
    a, b := p.c, q.c
    if len(a) < len(b) {
        a, b = b, a
    }
    r := make([], uint64, len(a))
    for i := range b {
        r[i] = f.add(a[i], b[i])
    }
    copy(r[len(b):], a[len(b):])
    return FpPoly{f, r}.trim()
}
```


13. 뺄셈은 피감수가 짧을 수도 있어 자리마다 있는 만큼만 꺼내 뺀다.

〈다항식 10〉 +≡

```
func (p FpPoly) Sub(q FpPoly) FpPoly {
    f := p.f
    n := max(len(p.c), len(q.c))
    r := make([]uint64, n)
    for i := range n {
        var a, b uint64
        if i < len(p.c) {
            a = p.c[i]
        }
        if i < len(q.c) {
            b = q.c[i]
        }
        r[i] = f.sub(a, b)
    }
    return FpPoly{f, r}.trim()
}
```

14. 부호 반전, 상수배, 상수항 더하기는 계수별 손질이다. *addConst* 는 Schoof 장에서 $3x^2 + A$ 같은 식을 만들 때 쓴다.

〈다항식 10〉 +≡

```
func (p FpPoly) Neg() FpPoly {
    f := p.f
    r := make([]uint64, len(p.c))
    for i, v := range p.c {
        r[i] = f.neg(v)
    }
    return FpPoly{f, r}
}

func (p FpPoly) MulScalar(a uint64) FpPoly {
    f := p.f
    a = f.reduce(a)
    r := make([]uint64, len(p.c))
    for i, v := range p.c {
        r[i] = f.mul(v, a)
    }
    return FpPoly{f, r}.trim()
}

func (p FpPoly) addConst(a uint64) FpPoly {
    r := p.clone()
    r.c[0] = p.f.add(r.c[0], p.f.reduce(a))
    return r
}
```

15. 곱셈은 두 별을 마련한다. 교과서식 $O(n^2)$ 곱셈과, 다음 절부터 펼쳐질 수론적 변환(NTT)의 $O(n \log n)$ 곱셈이다. 작은 다항식에는 교과서가 이긴다 — 변환의 상차림 비용이 본 요리보다 비싸기 때문이다. 문턱은 결과 길이 128로 잡았다. 반대쪽 끝의 안전판도 하나 둔다. 뒤에 보겠지만 우리 변환은 길이 2^{23} 까지만 지원하므로, 그보다 긴(이 프로그램에서는 나올 일 없는) 곱은 교과서로 되돌린다.

〈다항식 10〉 +=

```
const mulThreshold = 128 // 이보다 짧은 곱은 교과서식이 더 빠르다
func (p FpPoly) Mul(q FpPoly) FpPoly {
    n := len(p.c) + len(q.c) - 1
    if n < mulThreshold ∨ n > 1 << 23 {
        return p.mulSchool(q)
    }
    return FpPoly{p.f, p.mulNTT(q, n)}.trim()
}
```

16. 교과서식 곱셈. 0인 계수를 건너뛰는 것은 나눗셈 다항식처럼 듬성듬성한 피연산자에서 쏠쏠하다.

〈다항식 10〉 +=

```
func (p FpPoly) mulSchool(q FpPoly) FpPoly {
    f := p.f
    r := make([]uint64, len(p.c) + len(q.c) - 1)
    for i, ai := range p.c {
        if ai == 0 {
            continue
        }
        for j, bj := range q.c {
            if bj == 0 {
                continue
            }
            r[i+j] = f.add(r[i+j], f.mul(ai, bj))
        }
    }
    return FpPoly{f, r}.trim()
}
```

17. 수론적 변환. 빠른 곱셈의 사연은 1805 년으로 거슬러 오른다. 가우스는 소행성 궤도를 맞추다가 삼각급수 보간을 빨리 하는 법을 노트에 적어 두었는데, 그게 바로 고속 푸리에 변환(FFT)이다—푸리에가 열 방정식 논문을 내기 두 해 전이다. 노트는 사후 전집에 라틴어로 묻혀 있다가, 1965 년 Cooley 와 Tukey 가 같은 것을 재발견해 계산의 역사를 바꾼 뒤에야 재조명되었다. Tukey 의 동기가 재미있다: 소련 핵실험을 지진계로 감시하려면 긴 신호의 스펙트럼을 그때그때 계산해야 했던 것이다. 냉전이 알고리즘 하나를 캐냈다.

다항식 곱셈이 여기 걸리는 이유는 곱셈이 곧 계수열의 합성곱이고, 변환이 합성곱을 점별 곱으로 바꿔 주기 때문이다. 차수 n 다항식을 $2n$ 개 지점에서 값매김하고(변환), 값끼리 곱한 뒤(점별 곱 $O(n)$), 도로 보간하면(역변환) 곱이 나온다. 값매김·보간을 1의 2^k 제곱근들에서 하면 분할정복으로 $O(n \log n)$ — 이것이 FFT 곱셈이다.

다만 복소수 FFT 를 그대로 쓰면 부동소수점 오차가 계수에 스며든다. 우리 계수는 F_p 의 원소라 반올림을 한 톨도 용납할 수 없다. 다행히 FFT 에 필요한 것은 복소수 자체가 아니라 “1의 원시 2^k 제곱근이 있는 환”이라는 무대 장치뿐이다. $m = c \cdot 2^s + 1$ 꼴 소수의 F_m 이 정확히 그런 무대다: 곱셈군의 위수가 $m - 1 = c \cdot 2^s$ 이니 원시근 g 의 거듭제곱 $g^{(m-1)/2^k}$ 이 1의 원시 2^k 제곱근이 된다($k \leq s$). 이렇게 유한체에서 하는 FFT 를 수론적 변환(number-theoretic transform, NTT)이라 부른다. 오차는 원천 봉쇄, 셈은 전부 정수다.

18. 그런데 무대가 하나로는 모자란다. 진짜 계수—법으로 줄이기 전의 합성곱 값—는 최대 $n(p-1)^2$ 까지 자라는데, p 가 2^{63} 언저리면 이는 2^{149} 쯤 되어 어떤 64비트 법 m 으로도 담을 수 없다. 처방은 오래된 것이다: 서로소인 법 세 개로 각각 셈하고 중국인의 나머지 정리(CRT)로 합치면, 세 법의 곱 $m_1 m_2 m_3 \approx 2^{152.5}$ 미만의 값이 유일하게 복원된다. $n \leq 2^{23}$ 에서 $n(p-1)^2 < 2^{23} \cdot 2^{126} = 2^{149}$ 이므로 여유 있게 들어간다—앞서 *Mul* 이 2^{23} 에서 손을 떼 이유가 이것이다.

세 소수는 경시 프로그래밍판에서 오래 굴러 검증된 명물들로 골랐다. 각각의 원시근도 함께 적는다. $m - 1$ 이 2의 큰 거듭제곱을 인수로 가진다는 것, 즉 지원하는 변환 길이가 각각 2^{23} , 2^{56} , 2^{57} 이라는 것이 자격 요건이다(셋의 최솟값이 전체의 한계가 된다).

< 다항식 10 > +=

```
const (
    ntt1, ntt1g = 998244353, 3 // 119 · 223 + 1
    ntt2, ntt2g = 1945555039024054273, 5 // 27 · 256 + 1
    ntt3, ntt3g = 4179340454199820289, 3 // 29 · 257 + 1
)
```

19. 변환 본체는 제자리(in-place) 반복문 꼴의 Cooley–Tukey 다. 먼저 첨자를 비트 뒤집기 순서로 재배열하고, 길이 2, 4, 8, ..., n 의 나비(butterfly) 연산을 겹겹이 쌓는다. 나비 하나는 $(u, v) \mapsto (u + \omega^j v, u - \omega^j v)$ — 길이 2의 변환이다. 역변환은 회전 인자 ω 를 역원으로 같고 끝에 n^{-1} 을 곱하면 된다는 것이 변환의 잘 알려진 대칭이다.

〈 다항식 10〉 +=

```
func ntt(a []uint64, m, g uint64, invert bool) {
    n := len(a)
    〈첨자를 비트 뒤집기 순서로 재배열한다 20〉
    for length := 2; length ≤ n; length <<= 1 {
        w := powmod(g, (m - 1)/uint64(length), m)
        if invert {
            w = powmod(w, m - 2, m)
        }
        half := length >> 1
        for i := 0; i < n; i += length {
            wn := uint64(1)
            for j := i; j < i + half; j++ {
                u, v := a[j], mulmod(a[j + half], wn, m)
                a[j] = addmod(u, v, m)
                a[j + half] = submod(u, v, m)
                wn = mulmod(wn, w, m)
            }
        }
    }
    if invert {
        ninv := powmod(uint64(n), m - 2, m)
        for i := range a {
            a[i] = mulmod(a[i], ninv, m)
        }
    }
}
```

20. 재배열은 첨자 i 의 비트를 뒤집은 자리 j 와 맞바꾸는 것인데, j 를 매번 새로 뒤집지 않고 “이진수 덧셈을 최상위 비트부터 하는” 증가 트릭으로 이어간다. 뒤집힌 세계에서 1을 더하는 셈이다.

〈첨자를 비트 뒤집기 순서로 재배열한다 20〉 ≡

```
for i, j := 1, 0; i < n; i++ {
    bit := n >> 1
    for; j & bit ≠ 0; bit >>= 1 {
        j ⊕= bit
    }
    j |= bit
    if i < j {
        a[i], a[j] = a[j], a[i]
    }
}
```

이 코드는 19절에서 쓰인다.

21. 법 하나에서의 합성곱. 두 계수열을 길이 n 으로 늘려 변환하고, 점별로 곱하고, 역변환한다. 들어오는 계수는 p 로 줄어 있지 국소 법 m 으로 줄어 있지는 않으니 먼저 m 으로 줄인다(m_1 은 p 보다 작을 수 있다).

〈다항식 10〉 +=

```
func nttConv(a, b []uint64, n int, m, g uint64) []uint64 {
    fa := make([]uint64, n)
    for i, v := range a {
        fa[i] = v % m
    }
    fb := make([]uint64, n)
    for i, v := range b {
        fb[i] = v % m
    }
    ntt(fa, m, g, false)
    ntt(fb, m, g, false)
    for i := range fa {
        fa[i] = mulmod(fa[i], fb[i], m)
    }
    ntt(fa, m, g, true)
    return fa
}
```

22. NTT 곱셈의 지휘부. 결과 길이 이상의 2의 거듭제곱 n 을 잡고, 세 법에서 따로 합성곱을 구한 뒤 계수마다 CRT로 복원한다.

〈다항식 10〉 +=

```
func (p FpPoly) mulNTT(q FpPoly, rlen int) []uint64 {
    n := 1
    for n < rlen {
        n <= 1
    }
    r1 := nttConv(p.c, q.c, n, ntt1, ntt1g)
    r2 := nttConv(p.c, q.c, n, ntt2, ntt2g)
    r3 := nttConv(p.c, q.c, n, ntt3, ntt3g)
    〈Garner의 방법으로 세 나머지에서 계수를 복원한다 23〉
    return r
}
```

23. 복원은 Garner의 방법을 쓴다. 참값 $x < m_1 m_2 m_3$ 를 혼합기수 $x = a_1 + t_2 m_1 + t_3 m_1 m_2$ 꼴로 풀면

$$t_2 = (a_2 - a_1) m_1^{-1} \bmod m_2, \quad t_3 = (a_3 - (a_1 + t_2 m_1)) (m_1 m_2)^{-1} \bmod m_3$$

이고, 이 전개를 p 로 줄이며 그대로 읽으면 원하는 계수 $x \bmod p$ 다. 128비트 큰 수를 실제로 만들 필요가 전혀 없다는 것이 이 방법의 미덕이다 — 모든 셈이 *mulmod* 몇 번으로 끝난다. $m_1 < m_2 < m_3$ 이라 중간값들이 항상 해당 범 미만임도 눈여겨보라.

〈Garner의 방법으로 세 나머지에서 계수를 복원한다 23〉≡

```

inv12 := powmod(ntt1, ntt2 - 2, ntt2) // m1-1 mod m2
m12 := mulmod(ntt1, ntt2 % ntt3, ntt3) // m1m2 mod m3
inv123 := powmod(m12, ntt3 - 2, ntt3) // (m1m2)-1 mod m3
pm := p.f.p
m1p := ntt1 % pm
m12p := mulmod(m1p, ntt2 % pm, pm)
r := make([]uint64, rlen)
for i := range r {
    t2 := mulmod(submod(r2[i], r1[i], ntt2), inv12, ntt2)
    x12 := addmod(r1[i], mulmod(t2, ntt1, ntt3), ntt3)
    t3 := mulmod(submod(r3[i], x12, ntt3), inv123, ntt3)
    v := addmod(r1[i] % pm, mulmod(t2 % pm, m1p, pm), pm)
    r[i] = addmod(v, mulmod(t3 % pm, m12p, pm), pm)
}

```

이 코드는 22절에서 쓰인다.

24. 다항식 나눗셈과 그 친구들. 나눗셈은 초등학교 세로셈 그대로다: 남은 것의 최고차 항을 제수의 최고차 항으로 나눠 몫의 한 자리를 얻고, 그만큼 덜어 낸다. 제수가 모닉(최고차 계수 1)이면 자리마다 하던 역원 곱이 통째로 사라지므로 따로 우대한다—나눗셈 다항식 계산이 바로 이 우대의 단골손님이다.

〈 다항식 10 〉 $+=$

```
func (p FpPoly) DivMod(q FpPoly)(quo, rem FpPoly) {
    f := p.f
    if len(p.c) < len(q.c) {
        return f.Poly(0), p.clone()
    }
    qd := q.Deg()
    monic := q.c[qd] == 1
    var qInv uint64
    if !monic {
        qInv = f.inv(q.c[qd])
    }
    quoC := make([]uint64, len(p.c) - len(q.c) + 1)
    remC := append([]uint64(Λ), p.c...)
    〈 몫의 자리를 하나씩 정하며 나머지를 깎는다 25 〉
    return FpPoly{f, quoC}.trim(), FpPoly{f, remC}.trim()
}
```

25. 안쪽 고리. 나머지의 차수가 제수 아래로 떨어지면 끝난다.

〈 몫의 자리를 하나씩 정하며 나머지를 깎는다 25 〉 $\equiv$

```

for {
     $td := \text{len}(\text{rem}C) - 1$ 
     $rd := td - qd$ 
    if  $rd < 0 \vee (\text{len}(\text{rem}C) \equiv 1 \wedge \text{rem}C[0] \equiv 0)$  {
        break
    }
     $\text{coef} := \text{rem}C[td]$ 
    if  $\neg \text{monic}$  {
         $\text{coef} = f.\text{mul}(\text{coef}, q\text{Inv})$ 
    }
     $\text{quo}C[rd] = \text{coef}$ 
    for  $i, qi := \text{range } q.c$  {
        if  $qi \equiv 0$  {
            continue
        }
         $\text{rem}C[rd + i] = f.\text{sub}(\text{rem}C[rd + i], f.\text{mul}(qi, \text{coef}))$ 
    }
     $n := \text{len}(\text{rem}C)$ 
    for  $n > 1 \wedge \text{rem}C[n - 1] \equiv 0$  {
         $n--$ 
    }
     $\text{rem}C = \text{rem}C[:n]$ 
}

```

이 코드는 24 절에서 쓰인다.

26. 나머지만 필요할 때의 준말과, 모닉으로 만드는 손질.

〈 다항식 10 〉 $+ \equiv$

```

func ( $p$  FpPoly)  $\text{Mod}(q$  FpPoly) FpPoly {
     $-, r := p.\text{DivMod}(q)$ 
    return  $r$ 
}

func ( $p$  FpPoly)  $\text{Monic}()$  FpPoly {
     $d := p.\text{Deg}()$ 
    if  $p.c[d] \equiv 1$  {
        return  $p.\text{clone}()$ 
    }
    return  $p.\text{MulScalar}(p.f.\text{inv}(p.c[d]))$ 
}

```


27. 합동식 거듭제곱 $p^e \bmod m$. Schoof 알고리즘의 심장 박동이다 — 프로베니우스 $x \mapsto x^q$ 를 계산한다는 것이 곧 이 함수로 $e = q$ 를 먹이는 일이고, 그때마다 NTT 곱셈 예순 번쯤이 연달아 된다.

〈다항식 10〉 +≡

```
func (p FpPoly) PowMod(e uint64, m FpPoly) FpPoly {
    r := p.f.Poly(1)
    if e == 0 {
        return r
    }
    base := p.Mod(m)
    for e > 0 {
        if e & 1 == 1 {
            r = r.Mul(base).Mod(m)
        }
        e >>= 1
        if e > 0 {
            base = base.Mul(base).Mod(m)
        }
    }
    return r
}
```

28. 최대공약수는 유클리드 호제법이다. 기원전 300년의 알고리즘이 21세기의 암호 코드에 그대로 앉아 있다—이보다 오래 현역인 코드는 없다. 답은 상수배 차이가 남으니 모닉으로 다듬어 대표를 정한다.

〈다항식 10〉 +≡

```
func (p FpPoly) GCD(q FpPoly) FpPoly {
    a, b := p, q
    for !b.isZero() {
        a, b = b, a.Mod(b)
    }
    return a.Monic()
}
```

29. 법 h 에 대한 역원은 확장 유클리드로 구한다. 베주 계수 t 를 나란히 끌고 가면 마지막에 $tp \equiv r \pmod{h}$ 가 남는데, r 가 0 아닌 상수면 $p^{-1} = t/r$ 이고, r 의 차수가 남아 있으면 p 와 h 가 서로소가 아니라 역원이 없다. 실패를 *ok*로 알리는 까닭은 Schoof 장에서 밝혀진다—놀랍게도 이 “실패”가 거기서는 황재다.

〈다항식 10〉 +≡

```
func (p FpPoly) ModInverse(h FpPoly)(FpPoly, bool) {
    f := p.f
    r, newR := h, p.Mod(h)
    t, newT := f.Poly(0), f.Poly(1)
    for ¬newR.isZero() {
        quo, _ := r.DivMod(newR)
        r, newR = newR, r.Sub(quo.Mul(newR))
        t, newT = newT, t.Sub(quo.Mul(newT))
    }
    if r.Deg() > 0 {
        return f.Poly(0), false
    }
    return t.MulScalar(f.inv(r.c[0])), true
}
```

30. 시험대. 무작위 다항식을 만드는 손이 먼저 필요하다. 최고차 계수는 0이 되지 않게 한다.

〈elliptic_test.go 3〉 +≡

```
func randPoly(f *Fp, n int) FpPoly {
    c := make([]uint64, n)
    for i := range c {
        c[i] = rng.Uint64() % f.p
    }
    if c[n-1] == 0 {
        c[n-1] = 1
    }
    return FpPoly{f, c}
}
```

31. NTT 소수 세 개가 정말 명물인지부터 재확인한다. 소수성은 *big.Int* 의 Miller–Rabin 에게 묻고, 원시근은 “ $g^{(m-1)/q} \neq 1$ 이 $m-1$ 의 모든 소인수 q 에 대해 성립”이라는 판정으로 확인한다.

⟨elliptic_test.go 3⟩ +≡

```
func TestNTTModuli(t *testing.T) {
    cases := []struct {
        m, g uint64
        qs []uint64 // m-1 의 소인수들
    }{
        {ntt1, ntt1g, []uint64{2, 7, 17}},
        {ntt2, ntt2g, []uint64{2, 3}},
        {ntt3, ntt3g, []uint64{2, 29}},
    }
    for _, c := range cases {
        if ¬new(big.Int).SetUint64(c.m).ProbablyPrime(32) {
            t.Fatalf("%d은 소수가 아니다", c.m)
        }
        for _, q := range c.qs {
            if (c.m-1)%q ≠ 0 {
                t.Fatalf("%d은 %d-1 의 인수가 아니다", q, c.m)
            }
            if powmod(c.g, (c.m-1)/q, c.m) ≡ 1 {
                t.Fatalf("%d은 %d 의 원시근이 아니다", c.g, c.m)
            }
        }
    }
}
```

32. 곱셈 두 벌이 같은 답을 내는지—작은 체와 큰 체에서 각각—대조한다. 길이는 일부러 문턱 너머로 잡아 *Mul* 이 NTT 길로 가게 한다.

⟨elliptic_test.go 3⟩ +≡

```
func TestPolyMul(t *testing.T) {
    for _, p := range []uint64 { 97, p61 } {
        f := NewFp(p)
        a, b := randPoly(f, 230), randPoly(f, 179)
        if ¬a.Mul(b).equal(a.mulSchool(b)) {
            t.Fatalf("p=%d: NTT 곱셈과 교과서 곱셈이 다르다", p)
        }
    }
}
```

33. 나눗셈은 $a = qb + r$ 와 $\deg r < \deg b$ 를, 역원은 $aa^{-1} \equiv 1 \pmod{h}$ 를 확인한다.

`<elliptic_test.go 3> +=`

```
func TestPolyDiv(t *testing.T) {
    f := NewFp(10007)
    for range 50 {
        a := randPoly(f, 1 + int(rng.Uint64() % 40))
        b := randPoly(f, 1 + int(rng.Uint64() % 20))
        q, r := a.DivMod(b)
        if ¬r.isZero() ∧ r.Deg() ≥ b.Deg() {
            t.Fatal("나머지의 차수가 제수보다 크거나 같다")
        }
        if ¬q.Mul(b).Add(r).equal(a) {
            t.Fatal("a ≠ q*b + r")
        }
    }
}

func TestPolyInverse(t *testing.T) {
    f := NewFp(101)
    h := f.Poly(1, 1, 0, 0, 1) // x4 + x + 1
    for range 50 {
        a := randPoly(f, 1 + int(rng.Uint64() % 4))
        inv, ok := a.ModInverse(h)
        if ¬ok {
            continue
        }
        if ¬a.Mul(inv).Mod(h).equal(f.Poly(1)) {
            t.Fatal("a * a-1 ≠ 1 (mod h)")
        }
    }
}
```

34. 타원 곡선. 드디어 주인공이다. 체 K 위의 타원 곡선은

$$E : y^2 = x^3 + Ax + B, \quad 4A^3 + 27B^2 \neq 0$$

꼴 방정식의 해집합에 무한원점 $\mathcal{O}$ 를 하나 보탠 것이다(표수가 2나 3이 아닐 때는 어떤 삼차 곡선도 이 바이어슈트라스 표준형으로 옮길 수 있다). 조건 $4A^3 + 27B^2 \neq 0$ 은 판별식이 0이 아니라는 말로, 곡선이 스스로를 찌르거나(침점) 겹치지(교차점) 않고 매끈하다는 보증이다.

이름부터 짚고 가자. 타원 곡선은 타원이 아니다. 이름의 사연은 이렇다. 타원의 둘레를 구하려던 18세기 수학자들은 $\int dx/\sqrt{3}$ 차식 꼴의 적분 앞에서 멈춰 섰다—초등함수로는 안 풀리는 이 부류가 타원 적분이라는 이름을 얻었다. 아벨과 야코비가 그 역함수(타원 함수)를 연구했고, 타원 함수가 매개화하는 곡선이 타원 곡선으로 불리게 된 것이다. 말하자면 증조부의 직업이 성씨가 된 격이다. 혈통은 유서 깊다: 디오판토스의 《산술》에도 사실상의 타원 곡선 문제가 나오고, 페르마가 무한강하법을 버린 곳도 여기다. 20세기에는 모텔(Mordell)이 유리점들이 유한생성군을 이룸을 보였고, 와일스가 페르마의 마지막 정리를 무너뜨린 결정타도 타원 곡선의 모듈러성이었다. 그리고 1985년, Koblitz와 Miller가 서로 모르게 동시에 “이 군을 암호에 쓰자”고 제안하면서 이 곡선은 순수 수학의 귀족에서 만인의 호주머니(스마트폰의 TLS, 비트코인 지갑) 속 일꾼이 되었다.

35. 암호가 탐낸 것은 곡선 위의 점들이 이루는 군(group)이다. 군 법칙은 자와 컴퍼스로 그릴 수 있을 만큼 기하적이다: 두 점 P, Q 를 지나는 직선은 곡선과 반드시 세 번째 점 R 에서 만나고(삼차 방정식이니 근이 셋), $P+Q$ 는 그 R 를 x 축에 대해 반사한 점이다.

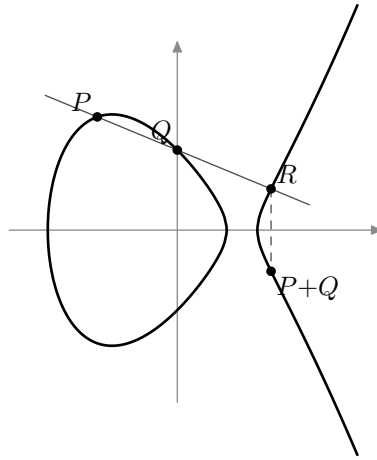


그림 1. $y^2 = x^3 - 2x + 1$ 위의 덧셈. 현이 곡선과 만나는 셋째 점 R 를 반사하면 $P+Q$ 다.

“셋째 교점을 그냥 합이라 하지, 왜 굳이 반사까지?”라는 물음이 당연히 나온다. 반사가 없으면 결합법칙이 깨진다. 반사 규칙 아래에서는 한 직선 위의 세 점이 언제나 $P+Q+R=\mathcal{O}$ 를 만족하는데, 이 대칭적인 진술이 군 공리를 모두 끌어낸다. 항등원은 무한원점 $\mathcal{O}$ —수직선이 곡선과 만나는 “하늘 끝의 점”이다. $P=(x,y)$ 의 역원은 수직으로 마주 보는 $-P=(x,-y)$ 이고, 결합법칙은... 직접 좌표로 확인하려 들면 종이 몇 장이 순식간에 사라진다. (제대로 하려면 사영 기하의 베주 정리나 복소 해석의 바이어슈트라스 $\wp$ 가 필요하다. 여기서는 시험 코드가 무작위 점 삼십 벌로 확인해 주는 것으로 만족하자—증명은 아니지만 잠은 잘 온다.)

이 프로그램은 점을 아핀 좌표 (x,y) 그대로 다루고 무한원점은 $(0,0)$ 으로 표기한다. $B \neq 0$ 인 곡선에서 $(0,0)$ 은 곡선 위에 없으므로 안심하고 쓸 수 있는 자리다(진지한 라이브러리는 나눗셈을 아끼려 사영 좌표를 쓰지만, 학습이 목적인 우리는 그림과 똑같이 생긴 공식이 더 값지다). 매개변수는 256비트 소수까지 감당해야 하니 이 층은 **big.Int** 다.

< 타원 곡선 35 > ≡

```
type Curve struct {
    P *big.Int // 바탕 체  $F_p$ 의 소수  $p$ 
    A, B *big.Int // 곡선 방정식  $y^2 = x^3 + Ax + B$ 의 계수
    Gx, Gy *big.Int // 밑점(생성원)  $G$ 
    N *big.Int //  $G$ 의 위수
    H *big.Int // 여인수(cofactor)
    BitSize int //  $p$ 의 비트 수
    Name string // 곡선의 통칭
}
```

이 코드는 36, 37, 38, 39, 40, 41, 42절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

36. 곡선 판정. 우변 $x^3 + Ax + B$ 는 여러 곳에서 쓰이니 *rhs*로 떼어 둔다. 관례대로 무한원점 $(0, 0)$ 에는 **false**를 답한다—“곡선 위의 점이나”는 물음에 “점이긴 한데 하늘에 있다”고 답할 수는 없으니.

〈 타원 곡선 35 〉 +=

```
func (c *Curve) rhs(x *big.Int) *big.Int {
    r := new(big.Int).Mul(x, x)
    r.Mul(r, x)
    r.Add(r, new(big.Int).Mul(c.A, x))
    r.Add(r, c.B)
    return r.Mod(r, c.P)
}

func (c *Curve) IsOnCurve(x, y *big.Int) bool {
    if x.Sign() < 0 ∨ x.Cmp(c.P) ≥ 0 ∨ y.Sign() < 0 ∨ y.Cmp(c.P) ≥ 0 {
        return false
    }
    y2 := new(big.Int).Mul(y, y)
    y2.Mod(y2, c.P)
    return c.rhs(x).Cmp(y2) == 0
}
```

37. 무한원점 판별과 역원. $-P$ 는 x 축 반사이니 y 의 부호만 같면 된다.

〈 타원 곡선 35 〉 +=

```
func isInf(x, y *big.Int) bool { return x.Sign() == 0 ∧ y.Sign() == 0 }

func (c *Curve) Neg(x, y *big.Int)(*big.Int, *big.Int) {
    ny := new(big.Int).Neg(y)
    ny.Mod(ny, c.P)
    return new(big.Int).Set(x), ny
}
```

38. 덧셈. 그림 1을 좌표로 옮기면 된다. $P = (x_1, y_1)$ 과 $Q = (x_2, y_2)$ 를 잇는 현의 기울기가 m 일 때, 직선 $y = m(x - x_1) + y_1$ 을 곡선 방정식에 넣으면 $x^3 - m^2x^2 + \dots = 0$ 이 되고, 세 근의 합이 m^2 이라는 비에타 공식에서

$$x_3 = m^2 - x_1 - x_2, \quad y_3 = m(x_1 - x_3) - y_1$$

이 나온다(y_3 의 끝에 반사가 이미 반영되어 있다). 특수한 경우가 셋이다: 한쪽이 $\mathcal{O}$ 면 다른 쪽이 답이고, $x_1 = x_2$ 인데 $y_1 + y_2 \equiv 0$ 이면 현이 수직이라 $\mathcal{O}$ 이며, 두 점이 같으면 현 대신 접선을 써야 하니 *Double*에 넘긴다.

〈 타원 곡선 35 〉 +=

```
func (c *Curve) Add(x1, y1, x2, y2 *big.Int)(*big.Int, *big.Int) {
    if isInf(x1, y1) {
        return new(big.Int).Set(x2), new(big.Int).Set(y2)
    }
    if isInf(x2, y2) {
        return new(big.Int).Set(x1), new(big.Int).Set(y1)
    }
    if x1.Cmp(x2) == 0 {
        s := new(big.Int).Add(y1, y2)
        if s.Mod(s, c.P).Sign() == 0 {
            return new(big.Int), new(big.Int)
        }
        return c.Double(x1, y1)
    }
    d := new(big.Int).Sub(x2, x1)
    d.Mod(d, c.P)
    d.ModInverse(d, c.P)
    m := new(big.Int).Sub(y2, y1)
    m.Mul(m, d).Mod(m, c.P)
    return c.chord(m, x1, y1, x2)
}
```

39. 기울기 m 에서 셋째 교점을 구해 반사하는 나머지는 덧셈과 두 배에서 똑같으니 함수로 나눠 가진다.

〈 타원 곡선 35 〉 +=

```
func (c *Curve) chord(m, x1, y1, x2 *big.Int)(*big.Int, *big.Int) {
    x3 := new(big.Int).Mul(m, m)
    x3.Sub(x3, x1).Sub(x3, x2).Mod(x3, c.P)
    y3 := new(big.Int).Sub(x1, x3)
    y3.Mul(y3, m).Sub(y3, y1).Mod(y3, c.P)
    return x3, y3
}
```


40. 두 배. Q 가 P 로 다가가는 극한에서 현은 접선이 된다. 곡선 방정식을 음함수 미분하면 $2yy' = 3x^2 + A$, 곧 접선의 기울기는 $m = (3x^2 + A)/2y$ 다.

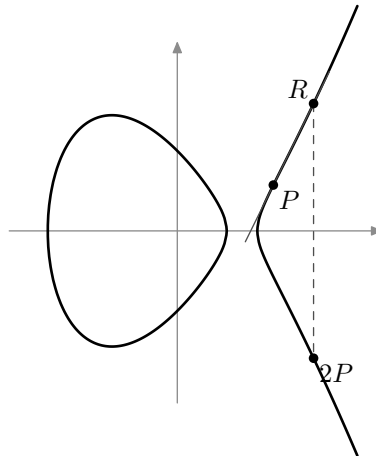


그림 2. 두 배. P 에서의 접선이 곡선과 다시 만나는 점 R 를 반사하면 $2P$ 다.

$y = 0$ 이면 접선이 수직이라 $2P = \mathcal{O}$ 다—그런 P 는 위수 2의 점이다. $(0, 0)$ 관례 덕에 이 검사가 무한원점까지 한꺼번에 처리한다.

〈 타원 곡선 35 〉 +=

```
func (c *Curve) Double(x, y *big.Int)(*big.Int, *big.Int) {
    if y.Sign() == 0 {
        return new(big.Int), new(big.Int)
    }
    d := new(big.Int).Lsh(y, 1)
    d.Mod(d, c.P)
    d.ModInverse(d, c.P)
    m := new(big.Int).Mul(x, x)
    m.Mul(m, big.NewInt(3)).Add(m, c.A)
    m.Mul(m, d).Mod(m, c.P)
    return c.chord(m, x, y, x)
}
```

41. 스칼라 곱 kP 가 암호의 심장이다. P 를 k 번 더하면 되지만 k 가 2^{256} 쯤 되면 우주의 나이로도 모자라다. 두 배 셈이 있으니 이진법이 구원한다: k 의 비트를 최상위부터 읽으며 “두 배, (비트가 1이면) 더하기”를 반복하면 $\log_2 k$ 걸음이다. 2^{256} 이 오백 걸음 남짓으로 준다—지수 함수를 상대로 거둔 로그의 승리이고, 뒤에 볼 이산 로그 문제의 어려움과 정확히 짝을 이루는 비대칭이다: 곱하기는 오백 걸음, 되돌리기는 2^{128} 걸음. k 의 부호는 무시하고 절댓값을 쓴다.

〈타원 곡선 35〉 +=

```
func (c *Curve) ScalarMult(px, py, k *big.Int)(*big.Int, *big.Int) {
    x, y := new(big.Int), new(big.Int)
    kk := new(big.Int).Abs(k)
    for i := kk.BitLen() - 1; i ≥ 0; i-- {
        x, y = c.Double(x, y)
        if kk.Bit(i) == 1 {
            x, y = c.Add(x, y, px, py)
        }
    }
    return x, y
}
```

42. 밑점 G 에 대한 준말과, ECDSA 검증이 쓰는 이중 스칼라 곱 $mG + nQ$.

〈타원 곡선 35〉 +=

```
func (c *Curve) ScalarBaseMult(k *big.Int)(*big.Int, *big.Int) {
    return c.ScalarMult(c.Gx, c.Gy, k)
}

func (c *Curve) CombinedMult(qx, qy, m, n *big.Int)(*big.Int, *big.Int) {
    x1, y1 := c.ScalarBaseMult(m)
    x2, y2 := c.ScalarMult(qx, qy, n)
    return c.Add(x1, y1, x2, y2)
}
```

43. 시험용 장난감 곡선은 암호 교과서의 고전인 F_{17} 위의 $y^2 = x^3 + 2x + 2$ 다. 점 $G = (5, 1)$ 의 위수는 19—군 전체가 열아홉 점짜리 순환군이라 손으로도 다 그려 볼 수 있는 크기다.

〈elliptic_test.go 3〉 +=

```
func toyCurve() *Curve {
    return &Curve{
        P: big.NewInt(17),
        A: big.NewInt(2),
        B: big.NewInt(2),
        Gx: big.NewInt(5),
        Gy: big.NewInt(1),
        N: big.NewInt(19),
        H: big.NewInt(1), BitSize: 5, Name: "toy17",
    }
}
```

44. 군다운지 심문한다: $19G = \mathcal{O}$, $18G = -G$, 그리고 무작위 점들로 교환·결합법칙.

```

< elliptic_test.go 3 > +=
func TestCurveGroup(t *testing.T) {
    c := toyCurve()
    if ¬c.IsOnCurve(c.Gx, c.Gy) {
        t.Fatal("G가 曲선 위에 없다")
    }
    x, y := c.ScalarBaseMult(c.N)
    if ¬isInf(x, y) {
        t.Fatalf("19G != 0", x, y)
    }
    x, y = c.ScalarBaseMult(big.NewInt(18))
    nx, ny := c.Neg(c.Gx, c.Gy)
    if x.Cmp(nx) ≠ 0 ∨ y.Cmp(ny) ≠ 0 {
        t.Fatal("18G != -G")
    }
    < 무작위 배수들로 교환법칙과 결합법칙을 확인한다 45 >
}

```

45. $P = iG$, $Q = jG$, $R = kG$ 를 뽑아 $P + Q = Q + P$ 와 $(P + Q) + R = P + (Q + R)$ 를 재 본다.

< 무작위 배수들로 교환법칙과 결합법칙을 확인한다 45 > ≡

```

for range 30 {
    pt := func()(*big.Int, *big.Int) {
        return c.ScalarBaseMult(big.NewInt(int64(rng.Uint64() % 19)))
    }
    px, py := pt()
    qx, qy := pt()
    rx, ry := pt()
    x1, y1 := c.Add(px, py, qx, qy)
    x2, y2 := c.Add(qx, qy, px, py)
    if x1.Cmp(x2) ≠ 0 ∨ y1.Cmp(y2) ≠ 0 {
        t.Fatal("P+Q != Q+P")
    }
    x1, y1 = c.Add(x1, y1, rx, ry)
    x2, y2 = c.Add(qx, qy, rx, ry)
    x2, y2 = c.Add(px, py, x2, y2)
    if x1.Cmp(x2) ≠ 0 ∨ y1.Cmp(y2) ≠ 0 {
        t.Fatal("(P+Q)+R != P+(Q+R)")
    }
}
}

```

이 코드는 44 절에서 쓰인다.

46. 나눗셈 다항식. Schoof 알고리즘으로 건너가기 전에 다리 하나를 놓아야 한다. E 의 n -등분점(n -torsion point), 곧 $nP = \mathcal{O}$ 인 점들을 통째로 붙드는 다항식이다. 나눗셈 다항식(division polynomial) $\psi_n \in F_p[x]$ 은 정확히 그 일을 한다: $\mathcal{O}$ 아닌 점 $P = (x, y)$ 에 대해

$$nP = \mathcal{O} \iff \psi_n(\xi) = \iota.$$

“점 P 를 n 으로 나누는 문제”에서 나온 이름이다. 홀수 n 에서 ψ_n 의 차수는 $(n^2 - 1)/2$ 인데, 이는 n -등분점이 $n^2 - 1$ 개($\mathcal{O}$ 제외)이고 x 좌표를 $\pm y$ 가 나눠 가진다는 사실과 정확히 맞아떨어진다. n^2 이라는 숫자에 잠깐 눈길을 주자—실수 위 곡선이라면 n 등분점이 고리 하나에 n 개뿐일 텐데, 복소수(그리고 유한체)의 타원 곡선은 도넛 모양이라 고리가 두 방향으로 있어 $n \times n$ 개가 된다. Schoof 알고리즘 비용의 대부분이 이 n^2 에서 나온다.

고맙게도 ψ_n 은 점화식으로 술술 나온다. $f = x^3 + Ax + B$ 라 할 때

$$\psi_{2m+1} = \psi_{m+2}\psi_m^3 - \psi_{m-1}\psi_{m+1}^3 \cdot (16f^2)^{\pm 1}, \quad \psi_{2m} = \frac{\psi_m}{\psi_2} (\psi_{m+2}\psi_{m-1}^2 - \psi_{m-2}\psi_{m+1}^2)$$

꿀이다(원래 ψ_n 은 n 이 짝수일 때 y 의 홀수 차수 항을 가지는데, $y^2 = f$ 로 짝수 차수만 남기고 y 한 개를 약속으로 떼어 둔 것이 위 식의 $16f^2$ 곱셈·나눗셈으로 나타난다). 셈은 *divPoly*가 하고, 재귀가 같은 항을 거듭 찾으므로 캐시를 둔다.

점 세기 엔진의 곡선은 **uint64** 체 위에 산다. **big.Int** 층의 **Curve**와 별개로 작은 곡선 타입을 하나 두고, 캐시도 여기에 딸려 보낸다—Schoof가 소수마다 고루틴을 띄울 때 곡선을 하나씩 나눠 가지면 잠금 없이 병렬이 된다.

< 나눗셈 다항식 46 > ≡

```
type fpCurve struct {
    f *Fp
    A, B uint64
    dp map[int64]FpPoly // 나눗셈 다항식 캐시
}

func (c *fpCurve) poly() FpPoly {
    return c.f.Poly(c.B, c.A, 0, 1) // x^3 + Ax + B
}
```

이 코드는 47 절에도 정의되어 있다.

이 코드는 2 절에서 쓰인다.

47. 밑돌 다섯 장은 손으로 놓는다. $\psi_0 = 0$, $\psi_1 = 1$ 이고

$$\psi_2 = 4f, \quad \psi_3 = 3x^4 + 6Ax^2 + 12Bx - A^2,$$

$$\psi_4 = (8x^6 + 40Ax^4 + 160Bx^3 - 40A^2x^2 - 32ABx - 8A^3 - 64B^2) \cdot f$$

(ψ_2 와 ψ_4 는 본디 $2y$, $4y(\cdots)$ 인데 위 약속대로 $y \mapsto y^2 = f$ 로 바꿔 둔 것이다).

〈 나눗셈 다항식 46 〉 $\equiv$

```
func (c *fpCurve) divPoly(n int64) FpPoly {
    f := c.f
    if c.dp == Λ {
        c.dp = make(map[int64]FpPoly)
    }
    if d, ok := c.dp[n]; ok {
        return d
    }
    cache := func(dp FpPoly) FpPoly { c.dp[n] = dp; return dp }
    A, B := c.A, c.B
    switch n {
    case 0:
        return cache(f.Poly(0))
    case 1:
        return cache(f.Poly(1))
    case 2:
        return cache(c.poly().MulScalar(4))
    case 3:
        〈  $\psi_3$ 를 만들어 돌려준다 48 〉
    case 4:
        〈  $\psi_4$ 를 만들어 돌려준다 49 〉
    }
    〈 점화식으로  $\psi_n$ 을 만들어 돌려준다 50 〉
}
```

48. 〈 ψ_3 를 만들어 돌려준다 48 〉 $\equiv$

```
a2 := f.mul(A, A)
return cache(f.Poly(
    f.neg(a2), // -A^2
    f.mul(f.fromInt(12), B), // 12B
    f.mul(f.fromInt(6), A), // 6A
    0,
    f.fromInt(3), // 3x^4
))
```

이 코드는 47절에서 쓰인다.

49. $\langle \psi_4$ 를 만들어 돌려준다 49 $\rangle \equiv$

```

a2 := f.mul(A, A)
a3 := f.mul(a2, A)
b2 := f.mul(B, B)
ab := f.mul(A, B)
c0 := f.sub(f.neg(f.mul(f.fromInt(8), a3)), f.mul(f.fromInt(64), b2))
return cache(f.Poly(
    c0, //  $-8A^3 - 64B^2$ 
    f.neg(f.mul(f.fromInt(32), ab)), //  $-32AB$ 
    f.neg(f.mul(f.fromInt(40), a2)), //  $-40A^2$ 
    f.mul(f.fromInt(160), B), //  $160B$ 
    f.mul(f.fromInt(40), A), //  $40A$ 
    0,
    f.fromInt(8), //  $8x^6$ 
).Mul(c.poly()))

```

이 코드는 47 절에서 쓰인다.

50. 점화식 부분. $m = \lfloor n/2 \rfloor$ 언저리의 다항식 다섯을 모아 조립한다. 홀수 $n = 2m+1$ 이면 $\psi_{m+2}\psi_m^3 - \psi_{m-1}\psi_{m+1}^3$ 인데, 두 항 중 짝수 첨자 쪽이 $(4f)$ 인수를 세제곱으로 세 개 들고 있으니 $16f^2$ 로 나눠 균형을 맞춘다(어느 쪽인지는 m 의 홀짝이 정한다). 짝수 $n = 2m$ 이면 전체를 ψ_2 로 나눈다. 나눗셈은 언제나 나누어떨어진다—점화식이 보증하는 정제성이라, 나머지는 버려도 좋은 것이 아니라 애초에 0 이다.

$\langle$ 점화식으로 ψ_n 을 만들어 돌려준다 50 $\rangle \equiv$

```

m := n/2
p2m := c.divPoly(m - 2)
p1m := c.divPoly(m - 1)
pm := c.divPoly(m)
pm1 := c.divPoly(m + 1)
pm2 := c.divPoly(m + 2)
var dp FpPoly
if n & 1 == 1 {
    den := c.poly().Mul(c.poly()).MulScalar(16)
    t1 := pm2.Mul(pm.Mul(pm).Mul(pm))
    t2 := p1m.Mul(pm1.Mul(pm1).Mul(pm1))
    if m & 1 == 0 {
        t1, _ = t1.DivMod(den)
    } else {
        t2, _ = t2.DivMod(den)
    }
    dp = t1.Sub(t2)
} else {
    dp = pm.Mul(pm2.Mul(p1m.Mul(p1m)).Sub(p2m.Mul(pm1.Mul(pm1))))
    dp, _ = dp.DivMod(c.divPoly(2))
}
return cache(dp)

```

이 코드는 47 절에서 쓰인다.

51. 차수 공식으로 조립을 검사한다. n 이 홀수면 $\deg \psi_n = (n^2 - 1)/2$ 다. 짝수는 표준 나눗셈 다항식이 인수 $2y$ 를 품는데, 이 구현은 그 y 를 $y^2 = f$ 로 갈아 $\psi_2 = 4f$ 처럼 두므로 x -차수가 $(n^2 - 4)/2$ 에 $\deg f = 3$ 이 더 붙어 $(n^2 + 2)/2$ 가 된다($n = 2$ 면 3, $n = 4$ 면 9).

`<elliptic_test.go 3> +≡`

```
func TestDivPolyDeg(t *testing.T) {
    c := &fpCurve{f: NewFp(10007), A: 3, B: 8}
    for n := int64(3); n ≤ 30; n++ {
        want := int((n * n - 1)/2)
        if n % 2 ≡ 0 {
            want = int((n * n + 2)/2)
        }
        if got := c.divPoly(n).Deg(); got ≠ want {
            t.Fatalf("deg_psi_%d=_%d, want_%d", n, got, want)
        }
    }
}
```

52. Schoof 알고리즘. 곡선 E/F_p 위의 점은 몇 개일까? 암호학자에게 이 물음은 한가한 호기심이 아니다—군의 위수 $\#E(F_p)$ 에 큰 소인수가 없으면 이산 로그가 Pohlig–Hellman에게 낱알이 쪼개지므로(다음 장), 곡선을 고르려면 먼저 세어야 한다. x 마다 $x^3 + Ax + B$ 가 제곱잉여인지 보면 $O(p)$ 에 셀 수 있지만, p 가 256비트면 우주가 먼저 식는다.

1933년 Hasse가 과녁을 크게 좁혀 두었다: $\#E(F_p) = p + 1 - t$ 로 쓰면

$$|t| \leq 2\sqrt{p}.$$

x 마다 해가 평균 하나쯤(제곱잉여일 확률 절반에 해가 둘)이라는 직관의 엄밀한 형태로, 오차항 t 를 프로베니우스의 자취(trace of Frobenius)라 부른다. 그래도 후보가 $4\sqrt{p}$ 개—여전히 지수적으로 많다.

1985년 René Schoof가 처음으로 다항 시간 알고리즘을 내놓았다. 발상은 CRT 식 분할이다: t 전체는 몰라도 $t \bmod \ell$ 은 작은 무대에서 알아낼 수 있다. 작은 소수 ℓ 을 곱이 $4\sqrt{p}$ 를 넘을 때까지 모아 각각에서 $t \bmod \ell$ 을 구하면, 중국인의 나머지 정리가 t 를 하나로 꿰어 준다. ℓ 은 $O(\log p)$ 개면 족하다. 뒷이야기 하나: 이 논문은 처음에 “우아하나 비실용적”이라는 평을 들었다. 큰 ℓ 에서 $\deg \psi_\ell = (\ell^2 - 1)/2$ 짜리 다항식을 주무르는 비용이 만만치 않았던 탓인데, Elkies와 Atkin이 ψ_ℓ 을 차수 $(\ell - 1)/2$ 의 인수로 갈아 끼우는 개량(SEA 알고리즘)을 내놓아 실용의 문턱을 넘었다. 이 문서는 원조 Schoof를 구현한다—학습에는 원형이 낫다.

53. 작은 무대의 주인공은 프로베니우스 사상 $\pi(x, y) = (x^p, y^p)$ 다. F_p 의 원소는 $a^p = a$ 로 꿈쩍 않지만(페르마), 확대체에 사는 등분점들은 π 가 뒤섞는다. 핵심 정리는 π 가 E 위에서 특성 방정식

$$\pi^2 - t\pi + p = 0$$

을 만족한다는 것이다. 이를 ℓ -등분점에 국한하면, $\bar{t} = t \bmod \ell$ 과 $\bar{p} = p \bmod \ell$ 에 대해 $\pi^2(P) + \bar{p}P = \bar{t}\pi(P)$ 가 된다. 좌변을 계산해 놓고 $\bar{t} = 0, 1, \dots, \ell - 1$ 을 하나씩 우변에 넣어 맞는 것을 찾으면 $t \bmod \ell$ 이 나온다.

“ ℓ -등분점에 국한”을 코드로 말하면 이렇다: 점의 좌표를 구체적 수가 아니라 잉여환

$$R = F_p[x]/(\psi_\ell(x))$$

의 원소로 다룬다. ψ_ℓ 로 나눈 나머지만 기억하는 세계에서는 “모든 ℓ -등분점에서 동시에 성립하는 등식”만 참이 되므로, R 에서의 등식 검사가 곧 원하는 국한이다. 등분점을 하나도 명시적으로 찾지 않고 전부를 한꺼번에 다루는 것—이것이 Schoof의 마술이다.

〈Schoof 알고리즘 53〉≡

```
type qring struct {
    h FpPoly // 법:  $\psi_\ell$  또는 그 인수
    f *Fp
}

func (qr *qring) reduce(p FpPoly) FpPoly { return p.Mod(qr.h) }
```

이 코드는 54, 55, 56, 57, 58, 59, 60, 64, 65, 66, 70절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

54. R 위의 “점”은 사상(endomorphism)이다. x 좌표는 R 의 원소 $a(x)$, y 좌표는 $b(x) \cdot y$ 꼴로 두되 y 는 장부에만 있는 유령으로 다룬다 — y^2 이 나올 때마다 곡선 방정식으로 $f(x) = x^3 + Ax + B$ 로 바꿔치기하면 y 는 식에서 영영 지수 1을 넘지 않으므로, b 만 저장하면 된다. 항등 사상은 $(x, 1 \cdot y)$, 프로베니우스는 $(x^p, f^{(p-1)/2} \cdot y)$ 다 ($y^p = y(y^2)^{(p-1)/2} = y f^{(p-1)/2}$).

$\mathcal{O}$ 는 Go의 Λ 로 나타낸다. 하늘의 점에 이만큼 어울리는 값도 없다.

〈Schoof 알고리즘 53〉 +=

```
type endo struct {
    qr *qring
    x, y FpPoly // 점 (x, y · y) — 뒤의 y가 장부 속 유령
}

func newEnd(qr *qring, x, y FpPoly) *endo {
    return &endo{qr, qr.reduce(x), qr.reduce(y)}
}

func endoEq(pe, qe *endo) bool {
    return pe.x.equal(qe.x) ^ pe.y.equal(qe.y)
}

func endoNeg(pe *endo) *endo {
    if pe ==  $\Lambda$  {
        return  $\Lambda$ 
    }
    return newEnd(pe.qr, pe.x, pe.y.Neg())
}
```

55. 사상들의 덧셈은 곡선 장의 현-접선 공식 그대로다—수 대신 다항식이 들어갈 뿐이다. 다만 기울기의 분모를 “나누기”가 문제다. R 는 체가 아니라 환이라 역원이 없을 수도 있다. 역원 계산이 실패하면 어떻게 하나?

여기서 이 알고리즘의 가장 아름다운 반전이 나온다. *ModInverse*가 실패했다는 것은 분모와 ψ_ℓ 의 최대공약수가 자명하지 않다는 뜻—곧 방금 ψ_ℓ 의 진약수를 공짜로 주웠다는 뜻이다. 그 인수로 법을 갈아 끼우면 무대는 좁아지고 셈은 빨라진다. 실패가 아니라 횡재다. 오류 값에 주운 인수를 실어 보내고, 바깥 고리가 법을 좁혀 처음부터 다시 돌게 한다.

〈Schoof 알고리즘 53〉 +=

```
type zeroDivError struct { factor FpPoly }

func (e *zeroDivError) Error() string { return "0으로 나누셈: 법의 인수를 발견" }
```

56. 서로 다른 두 사상의 덧셈. x 좌표가 같은데 y 까지 같으면 두 배로 넘기고, y 가 다르면(반드시 서로 반수다) $\mathcal{O}$ 다. 기울기는 $m = (b_2 - b_1)/(a_2 - a_1) \cdot y$ 꼴인데, x_3 에 쓰이는 m^2 에서 $y^2 = f$ 가 튀어나오므로 셋째 교점 공식이 $x_3 = f m^2 - a_1 - a_2$ 로 변형된다(m 은 y 를 떼 뒀만 저장). 인자 A 와 fx 는 여기서는 안 쓰이지만 *endoDouble* 과 서명을 맞춘다.

〈Schoof 알고리즘 53〉 +=

```
func endoAdd(pe, qe *endo, A uint64, fx FpPoly)(*endo, error) {
    if pe ≡ Λ {
        return qe, Λ
    }
    if qe ≡ Λ {
        return pe, Λ
    }
    qr := pe.qr
    a1, b1 := pe.x, pe.y
    a2, b2 := qe.x, qe.y
    if a1.equal(a2) {
        if b1.equal(b2) {
            return endoDouble(pe, A, fx)
        }
        return Λ, Λ
    }
    adif := a2.Sub(a1)
    inv, ok := adif.ModInverse(qr.h)
    if ¬ok {
        return Λ, &zeroDivError{adif}
    }
    m := qr.reduce(b2.Sub(b1).Mul(inv))
    m2 := qr.reduce(m.Mul(m))
    a3 := qr.reduce(fx.Mul(m2)).Sub(a1.Add(a2))
    b3 := qr.reduce(m.Mul(a1.Sub(a3))).Sub(b1)
    return newEnd(qr, a3, b3), Λ
}
```

57. 두 배. 접선 기울기 $(3a_1^2 + A)/(2b_1y)$ 의 분모에서도 y 를 f 로 바꿔치기한다.

⟨Schoof 알고리즘 53⟩ +=

```

func endoDouble(pe *endo, A uint64, fx FpPoly)(*endo, error) {
    if pe ≡  $\Lambda$  {
        return  $\Lambda$ ,  $\Lambda$ 
    }
    qr := pe.qr
    a1, b1 := pe.x, pe.y
    num := qr.reduce(a1.Mul(a1)).MulScalar(3).addConst(A)
    den := qr.reduce(b1.Mul(fx)).MulScalar(2)
    inv, ok := den.ModInverse(qr.h)
    if ¬ok {
        return  $\Lambda$ , &zeroDivError{den}
    }
    m := qr.reduce(num.Mul(inv))
    a3 := qr.reduce(fx.Mul(m.Mul(m))).Sub(a1.MulScalar(2))
    b3 := qr.reduce(m.Mul(a1.Sub(a3))).Sub(b1)
    return newEnd(qr, a3, b3),  $\Lambda$ 
}

```

58. 스칼라 곱은 곡선 장과 같은 두 배-덧셈이다. 여기서 n 은 $p \bmod \ell$ 이라 **uint64**로 족하다.

⟨Schoof 알고리즘 53⟩ +=

```

func endoScalarMul(pe *endo, n, A uint64, fx FpPoly)(*endo, error) {
    if n ≡ 0 {
        return  $\Lambda$ ,  $\Lambda$ 
    }
    re := newEnd(pe.qr, pe.x, pe.y)
    started := false
    for i := 63; i ≥ 0; i -- {
        var err error
        if started {
            if re, err = endoDouble(re, A, fx); err ≠  $\Lambda$  {
                return  $\Lambda$ , err
            }
        }
        if (n >> uint(i)) & 1 ≡ 1 {
            if ¬started {
                started = true // re가 이미  $1 \cdot pe$ 다
            } else if re, err = endoAdd(re, pe, A, fx); err ≠  $\Lambda$  {
                return  $\Lambda$ , err
            }
        }
    }
    return re,  $\Lambda$ 
}

```

59. $\ell = 2$ 는 따로 논다. $t \equiv \#E \equiv p + 1 - t \pmod{2}$ 에서 t 가 짝수인 것은 E 에 위수 2의 점이 있을 때, 곧 $f(x) = x^3 + Ax + B$ 가 F_p 에 근을 가질 때다. 삼차식이 근을 가지는지는 기약성 검사로 판별한다: $x^p - x$ 는 F_p 의 원소 전부를 근으로 가지는 다항식이므로, $\gcd(x^p - x, f) = 1$ 이면 근이 없어 $t \equiv 1$, 아니면 $t \equiv 0 \pmod{2}$ 다.

〈Schoof 알고리즘 53〉 +=

```
func irreducible(qr *qring, q uint64) bool {
    x := qr.f.Poly(0, 1)
    xq := x.PowMod(q, qr.h).Sub(x)
    return xq.GCD(qr.h).equal(qr.f.Poly(1))
}
```

60. $t \bmod \ell$ 을 구하는 본체다. 잉여환을 차리고, 프로베니우스와 그 제곱을 만들고, 특성 방정식의 우변 후보를 차례로 대 본다. 0으로 나눗셈이 인수를 물어다 주면 법을 줄혀 재시도한다.

〈Schoof 알고리즘 53〉 +=

```
var errNoCharPoly = errors.New("프로베니우스가 특성 방정식을 만족하지 않는다")
func traceMod(f *Fp, A, B uint64, ell int64)(int64, error) {
    c := &fpCurve{f: f, A: A, B: B}
    fx := c.poly()
    q := f.p
    if ell == 2 {
        if irreducible(&qring{fx, f}, q) {
            return 1, A
        }
        return 0, A
    }
    qr := &qring{c.divPoly(ell).Monic(), f}
    qModEll := q % uint64(ell)
    qHalf := q/2
    x := f.Poly(0, 1)
    var err error
    for {
        〈 오류를 살핀다: 인수를 주웠으면 법을 줄히고, 가망 없으면 접는다 61〉
        〈 프로베니우스  $\pi$ 와  $\pi^2$ 을 만든다 62〉
        〈 특성 방정식에  $\bar{t}$  후보를 대 본다 63〉
    }
}
```

61. 첫 순회에서는 err 가 Λ 이라 그냥 지나간다. 이후로는 0으로 나눗셈이 남긴 인수로 $h \leftarrow \gcd(h, \text{인수})$ 하고 다시 돈다.

〈 오류를 살핀다: 인수를 주웠으면 법을 줄히고, 가망 없으면 접는다 61〉 $\equiv$

```
var zd *zeroDivError
switch {
case errors.As(err, &zd):
    qr.h = qr.h.GCD(zd.factor)
case errors.Is(err, errNoCharPoly):
    return 0, err
}
err =  $\Lambda$ 
```

이 코드는 60 절에서 쓰인다.

62. $\pi = (x^q, f^{(q-1)/2}y)$ 이고, π^2 은 지수를 q^2 으로 키우는 대신 이미 썬한 x^q 과 y 부분 $f^{q/2}$ 를 다시 q 제곱, $q+1$ 제곱해 얻는다 — $(x^q)^q = x^{q^2}$ 이고 $f^{\lfloor q/2 \rfloor (q+1)} = f^{(q^2-1)/2}$ 이다. q^2 은 **uint64** 를 넘칠 수 있으니 이 우회가 멋이 아니라 필수다.

〈 프로베니우스 π 와 π^2 을 만든다 62〉 $\equiv$

```
xq := x.PowMod(q, qr.h)
yq := fx.PowMod(qHalf, qr.h)
pi := newEnd(qr, xq, yq)
pi2 := newEnd(qr, xq.PowMod(q, qr.h), yq.PowMod(q+1, qr.h))
```

이 코드는 60 절에서 쓰인다.

63. 좌변 $S = \pi^2 + \bar{q} \text{id}$ 를 만들고 $S = \bar{t} \pi$ 가 되는 $\bar{t}$ 를 찾는다. $S = \mathcal{O}$ 면 $\bar{t} = 0$, $S = \pm \pi$ 면 $\bar{t} = \pm 1$ 이고, 나머지는 π 를 거듭 더해 가며 맞춘다. 사상 연산이 err 를 내면 **continue**로 바깥 고리에 돌아가 법을 줄이고 재시도한다. 후보가 다 떨어지면 이론상 있을 수 없는 일이니 $errNoCharPoly$ 로 접는다.

〈 특성 방정식에 $\bar{t}$ 후보를 대 본다 63 〉 $\equiv$

```

    id := newEnd(qr, x, f.Poly(1))
    var Q, S *endo
    if Q, err = endoScalarMul(id, qModEll, A, fx); err ≠ Λ {
        continue
    }
    if S, err = endoAdd(pi2, Q, A, fx); err ≠ Λ {
        continue
    }
    if S ≡ Λ {
        return 0, Λ
    }
    if endoEq(S, pi) {
        return 1, Λ
    }
    if endoEq(endoNeg(S), pi) {
        return -1, Λ
    }
    P := newEnd(qr, pi.x, pi.y)
    for t := int64(2); t < ell - 1; t++ {
        if P, err = endoAdd(P, pi, A, fx); err ≠ Λ {
            break
        }
        if endoEq(P, S) {
            return t, Λ
        }
    }
    if err ≡ Λ {
        err = errNoCharPoly
    }

```

이 코드는 60절에서 쓰인다.

64. 남은 것은 지휘부다. 그 전에 연장 두 개—다음 소수와 중국인의 나머지 정리. *NextPrime* 은 Miller–Rabin 판정(*ProbablyPrime*)으로 홀수를 걸러 올라간다.

〈Schoof 알고리즘 53〉 +≡

```

func NextPrime(n *big.Int) *big.Int {
    if n.Cmp(big.NewInt(2)) < 0 {
        return big.NewInt(2)
    }
    if n.Cmp(big.NewInt(2)) ≡ 0 {
        return big.NewInt(3)
    }
    p := new(big.Int).Set(n)
    if p.Bit(0) ≡ 0 {
        p.Add(p, big.NewInt(1))
        if p.ProbablyPrime(20) {
            return p
        }
    }
    for {
        p.Add(p, big.NewInt(2))
        if p.ProbablyPrime(20) {
            return p
        }
    }
}

```

65. 손자(孫子)의 《손자산경》에 “셋씩 세면 둘 남고 다섯씩 세면 셋 남고 일곱씩 세면 둘 남는 물건”을 묻는 문제가 있다—답은 23이고, 서양이 이를 중국인의 나머지 정리라 부르는 연유다. 서로소인 법 n_i 들에 대한 잉여 a_i 들에서 $x \bmod \prod n_i$ 를 복원한다. $q_i = \prod n/n_i$ 의 역원을 확장 유클리드(GCD)로 얻는 표준 공식이다.

〈Schoof 알고리즘 53〉 +=

```
func CRT(a, n []*big.Int) *big.Int {
    if a ≡ Λ ∨ n ≡ Λ {
        return Λ
    }
    prod := big.NewInt(1)
    for _, x := range n {
        prod.Mul(prod, x)
    }
    var c, q, s, z big.Int
    for i, x := range n {
        q.Div(prod, x)
        z.GCD(Λ, &s, x, &q)
        if z.Int64() ≠ 1 {
            return Λ
        }
        c.Add(&c, s.Mul(a[i], s.Mul(&s, &q)))
    }
    return c.Mod(&c, prod)
}
```

66. 지휘부 Schoof 는 Curve 의 메서드다. 점 세기 엔진이 uint64 체라 $p < 2^{61}$ 만 받는다($q+1$ 제곱 같은 지수가 uint64 안에 머물도록 여유를 두었다). 60비트 소수의 곡선이면 초 단위로 세어 준다—*math/big* 으로 짚던 옛 구현이 며칠 밤을 새우던 크기다.

〈Schoof 알고리즘 53〉 +=

```
func (c *Curve) Schoof() (*big.Int, error) {
    if c.P ≡ Λ ∨ c.P.Sign() ≤ 0 ∨ c.P.BitLen() > 61 {
        return Λ, errors.New("elliptic: Schoof 는 0 ≤ p < 2^61 이 필요하다")
    }
    p := c.P.Uint64()
    f := NewFp(p)
    A := reduceBig(c.A, p)
    B := reduceBig(c.B, p)
    〈 곱이 하세 구간을 덮을 때까지 소수 ℓ 을 모은다 67 〉
    〈 소수 ℓ 마다 고루틴 하나가 t mod ℓ 을 구한다 68 〉
    〈 CRT 로 t 를 복원하고 #E = p + 1 - t 를 낸다 69 〉
}
```


67. 하세 구간의 길이가 $4\sqrt{p}$ 이므로 $\prod \ell$ 이 그보다 커야 t 가 유일하게 잡힌다. $big.Int$ 의 $Sqrt$ 는 내림이니 1을 더해 안전쪽으로 잡는다.

〈 곱이 하세 구간을 덮을 때까지 소수 ℓ 을 모은다 67〉 $\equiv$

```
fsq := new(big.Int).Sqrt(c.P)
fsq.Add(fsq, big.NewInt(1)).Mul(fsq, big.NewInt(4))
var ells []*big.Int
M := big.NewInt(1)
for l := big.NewInt(2); M.Cmp(fsq) ≤ 0; l = NextPrime(l) {
    ells = append(ells, new(big.Int).Set(l))
    M.Mul(M, l)
}
```

이 코드는 66절에서 쓰인다.

68. ℓ 끼리는 완전히 독립이니 병렬은 공짜다. $traceMod$ 가 곡선과 캐시를 제 것으로 만들므로 공유하는 것은 불변인 체 f 뿐—잠금이 필요 없다.

〈 소수 ℓ 마다 고루틴 하나가 $t \bmod \ell$ 을 구한다 68〉 $\equiv$

```
traces := make([]*big.Int, len(ells))
errs := make([]error, len(ells))
var wg sync.WaitGroup
for i := range ells {
    wg.Add(1)
    go func() {
        defer wg.Done()
        t, err := traceMod(f, A, B, ells[i].Int64())
        if err ≠ Λ {
            errs[i] = err
            return
        }
        traces[i] = big.NewInt(t)
    }()
}
wg.Wait()
for _, err := range errs {
    if err ≠ Λ {
        return Λ, err
    }
}
```

이 코드는 66절에서 쓰인다.

69. CRT는 $[0, M)$ 의 대표를 주지만 하세의 t 는 음수일 수 있으니, $M/2$ 를 넘으면 M 을 빼서 대칭 구간으로 옮긴다.

〈CRT로 t 를 복원하고 $\#E = p + 1 - t$ 를 낸다 69〉 $\equiv$

```

t := CRT(traces, ells)
if t.Cmp(new(big.Int).Rsh(M, 1)) ≥ 0 {
    t.Sub(t, M)
}
res := new(big.Int).Sub(c.P, t)
return res.Add(res, big.NewInt(1)), Λ

```

이 코드는 66절에서 쓰인다.

70. 음수일 수 있는 $big.Int$ 계수를 $[0, p)$ 로 데려오는 잔심부름.

〈Schoof 알고리즘 53〉 $+=$

```

func reduceBig(v *big.Int, p uint64) uint64 {
    P := new(big.Int).SetUint64(p)
    return new(big.Int).Mod(v, P).Uint64()
}

```

71. 〈elliptic_test.go 3〉 $+=$

```

func naiveCount(p, a, b uint64) *big.Int {
    f := NewFp(p)
    isQR := make([]bool, p)
    for x := uint64(0); x < p; x++ {
        isQR[f.mul(x, x)] = true
    }
    n := uint64(1) // 무한원점
    for x := uint64(0); x < p; x++ {
        r := f.add(f.mul(f.mul(x, x), x), f.add(f.mul(a, x), b))
        if r == 0 {
            n++
        } else if isQR[r] {
            n += 2
        }
    }
    return new(big.Int).SetUint64(n)
}

```

72. 시험은 삼심제다. 1심: 작은 소수들에서 무식한 전수 세기와 대조한다. 전수 세기는 제공잉여 표를 만들어 x 마다 $f(x)$ 가 표에 있는지 본다 — $O(p)$ 시간, $O(p)$ 메모리의 정직한 셈이다.

(elliptic_test.go 3) +≡

```
func TestSchoofSmall(t *testing.T) {
    for _, p := range []uint64 { 7, 11, 101, 1009, 10007 } {
        f := NewFp(p)
        for range 3 {
            a, b := rng.Uint64() % p, rng.Uint64() % p
            disc := f.add(f.mul(4, f.mul(f.mul(a, a), a)), f.mul(27, f.mul(b, b)))
            if disc == 0 {
                continue // 특이 곡선은 타원 곡선이 아니다
            }
            c := &Curve{
                P: new(big.Int).SetUint64(p),
                A: new(big.Int).SetUint64(a),
                B: new(big.Int).SetUint64(b),
            }
            got, err := c.Schoof()
            if err != Λ {
                t.Fatal(err)
            }
            if want := naiveCount(p, a, b); got.Cmp(want) != 0 {
                t.Fatalf("p=%d a=%d b=%d: Schoof=%v, 전수=%v", p, a, b, got, want)
            }
        }
    }
}
```

73. 2심: 전수 세기가 아직 가능한 중간 크기 $p = 1000003$.

(elliptic_test.go 3) +≡

```
func TestSchoofMedium(t *testing.T) {
    const p = 1000003
    c := &Curve{
        P: big.NewInt(p),
        A: big.NewInt(31337),
        B: big.NewInt(271828),
    }
    got, err := c.Schoof()
    if err != Λ {
        t.Fatal(err)
    }
    if want := naiveCount(p, 31337, 271828); got.Cmp(want) != 0 {
        t.Fatalf("Schoof=%v, 전수=%v", got, want)
    }
}
```

74. 3심: 전수 세기가 무리인 메르센 소수 $p = 2^{31} - 1$. 이제 대조할 답이 없으니 라그랑주 정리를 판사로 모신다—위수 n 이 맞다면 곡선 위의 어떤 점이든 $nP = \mathcal{O}$ 여야 한다. $p \equiv 3 \pmod{4}$ 라 제곱근이 $r^{(p+1)/4}$ 로 바로 나오니 점 찾기도 쉽다.

`<elliptic_test.go 3> +≡`

```
func TestSchoofOrder(t *testing.T) {
    const p = 2147483647 // 231 - 1
    c := &Curve{P: big.NewInt(p), A: big.NewInt(7), B: big.NewInt(11)}
    n, err := c.Schoof()
    if err != Λ {
        t.Fatal(err)
    }
    f := NewFp(p)
    var px, py uint64
    for x := uint64(1); ; x++ {
        r := f.add(f.mul(f.mul(x, x), x), f.add(f.mul(7, x), 11))
        if y := f.pow(r, (p+1)/4); f.mul(y, y) ≡ r {
            px, py = x, y
            break
        }
    }
    X, Y := c.ScalarMult(new(big.Int).SetUint64(px), new(big.Int).SetUint64(py), n)
    if ¬isInf(X, Y) {
        t.Fatalf("#E=%v 인데 nP ≠ 0", n)
    }
}
```

75. 이산 로그 문제. 암호의 판을 뒤집어 볼 차례다. 밀점 P 와 그 배수 $H = kP$ 가 주어졌을 때 k 를 찾는 것이 타원 곡선 이산 로그 문제(ECDLP)다. “로그”라는 이름은 곱셈군 버전 $g^k = h$ 에서 $k = \log_g h$ 인 데서 왔다—우리 군은 덧셈으로 쓰니 사실 나뉘셈이지만, 이름은 이미 굳었다. 앞 장에서 본 비대칭을 기억하자: k 에서 kP 로는 오백 걸음, 거꾸로는 (알려진 최선이) $\sqrt{N}$ 걸음. 이 낙차가 타원 곡선 암호의 밀천이고, 이 장은 그 “알려진 최선”들을 직접 만들어 밀천의 크기를 손으로 재 본다. 모두 c 의 밀점 위수 $c.N$ 이 아니라 임의의 위수 n 을 받는 안짜 함수로 만드는데, 마지막 손님 Pohlig–Hellman이 부분군들에서 재활용하기 위해서다.

76. 첫 손님은 Shanks의 아기걸음-거인걸음(baby-step giant-step)이다. 주인공 Daniel Shanks는 학위 없이 해군 연구소에서 일하다 마흔네에 박사가 된 늦깎이로, 원주율을 십만 자리까지 처음 계산한 사람이기도 하다. 1971년의 이 알고리즘은 시간-공간 맞바꿈의 원형이다. $m = \lceil \sqrt{n} \rceil$ 로 잡고 $k = a + mb$ ($1 \leq a \leq m$, $0 \leq b \leq m$)로 쪼개면

$$H = (a + mb)P \iff H - b(mP) = aP.$$

좌변을 모르니 우변을 몽땅 준비한다: 아기걸음으로 aP 들을 사전에 담아 두고(m 개), 거인걸음으로 H , $H - mP$, $H - 2mP, \dots$ 를 성큼성큼 밟으며 사전에 있나 물으면 된다. $O(\sqrt{n})$ 시간에 $O(\sqrt{n})$ 공간—사전 열쇠는 좌표를 문자열로 이어 붙여 만든다.

〈 이산 로그 문제 76 〉 ≡

```
func ptKey(x, y *big.Int) string { return x.Text(62) + "," + y.Text(62) }
func (c *Curve) Shank(px, py, hx, hy *big.Int) *big.Int {
    return c.shank(px, py, hx, hy, c.N)
}
func (c *Curve) shank(px, py, hx, hy, n *big.Int) *big.Int {
    m := new(big.Int).Sqrt(n)
    m.Add(m, big.NewInt(1))
    〈 아기걸음:  $aP$ 를 사전에 담는다 77 〉
    〈 거인걸음:  $H - b(mP)$ 가 사전에 있는지 밟아 본다 78 〉
    return  $\Lambda$ 
}
```

이 코드는 79, 83, 84, 85 절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

77. 〈 아기걸음: aP 를 사전에 담는다 77 〉 ≡

```
baby := make(map[string]*big.Int)
rx, ry := new(big.Int), new(big.Int)
for a := big.NewInt(1); a.Cmp(m) ≤ 0; a.Add(a, big.NewInt(1)) {
    rx, ry = c.Add(rx, ry, px, py)
    baby[ptKey(rx, ry)] = new(big.Int).Set(a)
}
```

이 코드는 76 절에서 쓰인다.

78. 걸음마다 $-mP$ 를 더하면 $H - b(mP)$ 가 차례로 나온다. 맞았다면 $k = a + mb$.

〈거인걸음: $H - b(mP)$ 가 사전에 있는지 봐야 본다 78〉 $\equiv$

```

     $sx, sy := c.Neg(px, py)$ 
     $sx, sy = c.ScalarMult(sx, sy, m)$ 
     $rx, ry = new(big.Int).Set(hx), new(big.Int).Set(hy)$ 
    for  $b := new(big.Int); b.Cmp(m) \leq 0; b.Add(b, big.NewInt(1))$  {
        if  $a, ok := baby[ptKey(rx, ry)]; ok$  {
            return  $new(big.Int).Add(a, new(big.Int).Mul(m, b))$ 
        }
         $rx, ry = c.Add(rx, ry, sx, sy)$ 
    }

```

이 코드는 76절에서 쓰인다.

79. 둘째 손님, Pollard의 ρ . 사전이 $\sqrt{n}$ 개나 되는 것이 아까웠던 John Pollard가 1978년에 공간을 상수로 줄였다. 발상은 생일 역설이다: 군 안에서 “충분히 무작위한” 걸음을 걸으면 $O(\sqrt{n})$ 걸음 안에 같은 점을 두 번 밟을 공산이 크다. 걸음의 궤적은 꼬리 달린 고리—그리스 문자 ρ 모양이라 이름이 ρ 다. 걸음마다 현재 점을 $aP + bH$ 꼴로 장부에 적어 두면, 충돌 $a_1P + b_1H = a_2P + b_2H$ 에서

$$k \equiv (a_1 - a_2)(b_2 - b_1)^{-1} \pmod{n}$$

이 나온다.

걸음 함수는 x 좌표를 3으로 나눈 나머지로 갈림길을 정한다: P 를 더하거나, 두 배 하거나, H 를 더한다. 어느 갈래든 장부(a, b)를 같이 갱신한다. 이 함수가 진짜 무작위는 아니지만 무작위 흉내로는 충분하다는 것이 경험칙이다.

〈 이산 로그 문제 76 〉 +=

```
func (c *Curve) PollardRho(px, py, hx, hy *big.Int) *big.Int {
    return c.pollardRho(px, py, hx, hy, c.N)
}

func (c *Curve) pollardRho(px, py, hx, hy, n *big.Int) *big.Int {
    step := func(x, y, a, b *big.Int)(*big.Int, *big.Int, *big.Int, *big.Int) {
        switch new(big.Int).Mod(x, big.NewInt(3)).Int64() {
        case 0:
            x, y = c.Add(x, y, px, py)
            a = new(big.Int).Add(a, big.NewInt(1))
            a.Mod(a, n)
        case 1:
            x, y = c.Double(x, y)
            a = new(big.Int).Lsh(a, 1)
            a.Mod(a, n)
            b = new(big.Int).Lsh(b, 1)
            b.Mod(b, n)
        default:
            x, y = c.Add(x, y, hx, hy)
            b = new(big.Int).Add(b, big.NewInt(1))
            b.Mod(b, n)
        }
    }
    return x, y, a, b
}

〈 무작위 출발점에서 토끼와 거북을 달리게 한다 80 〉
return  $\Lambda$ 
}
```

80. 충돌 감지는 Floyd의 토끼와 거북이다: 거북은 한 걸음, 토끼는 두 걸음. 고리에 들어서면 토끼가 거북을 반드시 따라잡는다—같은 점을 몽땅 저장하는 대신 둘만 기억하는 것이 공간 $O(1)$ 의 비결이다. 궤적이 불운하면(이를테면 $b_1 = b_2$ 거나 분모가 n 과 서로소가 아니면) 새 출발점으로 다시 달린다. 기대 걸음 수는 $\sqrt{\pi n/2}$ 쯤이니 여유를 두어 $8\sqrt{n}$ 걸음까지 달리고 접는다.

〈 무작위 출발점에서 토끼와 거북을 달리게 한다 80 〉 ≡

```

limit := new(big.Int).Sqrt(n)
limit.Mul(limit, big.NewInt(8)).Add(limit, big.NewInt(16))
li := int64(1) << 62
if limit.IsInt64() {
    li = limit.Int64()
}
for range 64 {
    〈 무작위  $a_1P + b_1H$ 에서 출발한다 81 〉
    x2, y2, a2, b2 := x1, y1, a1, b1
    for j := int64(0); j < li; j++ {
        x1, y1, a1, b1 = step(x1, y1, a1, b1)
        x2, y2, a2, b2 = step(step(x2, y2, a2, b2))
        if x1.Cmp(x2) == 0 ∧ y1.Cmp(y2) == 0 {
            〈 충돌 장부에서  $k$ 를 풀어 본다; 성공이면 답한다 82 〉
            break
        }
    }
}

```

이 코드는 79절에서 쓰인다.

81. 〈 무작위 $a_1P + b_1H$ 에서 출발한다 81 〉 ≡

```

a1, _ := rand.Int(rand.Reader, n)
b1, _ := rand.Int(rand.Reader, n)
vx, vy := c.ScalarMult(px, py, a1)
wx, wy := c.ScalarMult(hx, hy, b1)
x1, y1 := c.Add(vx, vy, wx, wy)

```

이 코드는 80절에서 쓰인다.

82. 분모 $b_2 - b_1$ 의 역원이 없으면(n 이 합성수면 생긴다) 이 궤적은 버린다. 풀린 k 는 $kP = H$ 로 검산하고 내보낸다.

〈충돌 장부에서 k 를 풀어 본다; 성공이면 답한다 82〉 $\equiv$

```

den := new(big.Int).Sub(b2, b1)
den.Mod(den, n)
if den.Sign()  $\equiv$  0  $\vee$  den.ModInverse(den, n)  $\equiv$   $\Lambda$  {
    break
}
k := new(big.Int).Sub(a1, a2)
k.Mul(k, den).Mod(k, n)
tx, ty := c.ScalarMult(px, py, k)
if tx.Cmp(hx)  $\equiv$  0  $\wedge$  ty.Cmp(hy)  $\equiv$  0 {
    return k
}

```

이 코드는 80절에서 쓰인다.

83. 셋째 손님 앞에 연장이 하나 필요하다: 정수의 소인수분해다. 여기서도 Pollard의 ρ 가 나온다—같은 생일 역설을 $x \mapsto x^2 + 1 \bmod n$ 의 궤적에 적용하면 n 의 소인수 q 를 $O(\sqrt{q})$ 에 뽑아내는 인수분해법이 된다(궤적을 q 로 줄여 보면 더 일찍 고리에 드는데, 그 순간이 $\gcd(x_i - x_j, n) > 1$ 로 드러난다). 한 사람이 같은 곡조로 두 문제를 푼 셈이다. 뽑아낸 인수가 소수라는 보장은 없으니— ρ 는 $12 = 2 \cdot 6$ 처럼 아무 약수나 물어 온다—재귀로 양쪽을 마저 쪼갬다. 소수를 만나면 목록에 없는다.

〈이산 로그 문제 76〉 $+\equiv$

```

func factorize(n *big.Int) []*big.Int {
    var factors []*big.Int
    var split func(m *big.Int)
    split = func(m *big.Int) {
        if m.Cmp(big.NewInt(1))  $\equiv$  0 {
            return
        }
        if m.ProbablyPrime(20) {
            factors = append(factors, m)
            return
        }
        d := rhoFactor(m)
        split(d)
        split(new(big.Int).Div(m, d))
    }
    split(new(big.Int).Set(n))
    return factors
}

```

84. 인수 하나를 찾는 ρ 본체. 거북(xs)을 박아 두고 토끼(x)를 달리게 한 뒤 주기적으로 갱신하는 Floyd 변형이다. 걸음 함수 $x \mapsto x^2 + c$ 의 상수 c 를 잘못 골라 궤적이 통째로 한 고리에 갇히면(5^k 같은 소수 거듭제곱의 단골 사고다) 자명한 인수 n 만 나오는데, 그럴 땐 c 를 갈아 다시 달린다. 짝수는 ρ 가 서투러 2를 먼저 벗겨 넘긴다. 합성수에는 반드시 $\sqrt{n}$ 이하의 소인수가 있으니 언젠가 걸린다.

(이산 로그 문제 76) +=

```
func rhoFactor(n *big.Int) *big.Int {
    if n.Bit(0) == 0 {
        return big.NewInt(2)
    }
    one := big.NewInt(1)
    for c := int64(1); ; c++ {
        xs := big.NewInt(2)
        x := big.NewInt(2)
        factor := big.NewInt(1)
        cycle := uint64(2)
        for factor.Cmp(one) == 0 {
            for j := uint64(0); j < cycle ^ factor.Cmp(one) == 0; j++ {
                x.Mul(x, x).Add(x, big.NewInt(c)).Mod(x, n)
                factor.GCD(1, new(big.Int).Sub(x, xs), n)
            }
            cycle <= 1
            xs.Set(x)
        }
        if factor.Cmp(n) != 0 {
            return factor // 자명하지 않은 인수를 잡았다
        }
    }
}
```

85. 셋째 손님은 Pohlig–Hellman 이다. 1978년, Diffie–Hellman 논문의 잉크가 마르기도 전에 나온 이 공격의 전언은 서늘하다: 이산 로그의 어려움은 군 위수 n 이 아니라 n 의 가장 큰 소인수가 정한다. $n = \prod q_i$ (q_i 는 서로소인 소수 거듭제곱)로 쪼개면, $t = n/q_i$ 배 한 점들 $P' = tP$, $H' = tH$ 는 위수 q_i 의 부분군에 살고 거기서 $k \bmod q_i$ 가 풀린다. 조각들은 CRT가 꿰매 준다. 각 조각은 Shanks나 ρ 로 $\sqrt{q_i}$ 걸음이니, n 이 매끄러우면(smooth—작은 소인수뿐이면) 전체가 와르르 무너진다. 실무에서 곡선의 위수를 소수로 고르는 이유가 정확히 이것이다 — secp256k1의 N 도 소수다.

(이산 로그 문제 76) +=

```
func (c *Curve) PohligHellman(px, py, hx, hy *big.Int) *big.Int {
    N := c.N
    ( 위수를 소수 거듭제곱들로 쪼갬다 86 )
    ( 부분군마다 작은 이산 로그를 푼다 87 )
    return CRT(dLogs, qs)
}
```

86. *factorize* 가 준 소인수 목록을 정렬해 같은 소수끼리 거듭제곱으로 뭉친다.

〈 위수를 소수 거듭제곱들로 쪼갬다 86〉 ≡

```

factors := factorize(new(big.Int).Set(N))
sort.Slice(factors, func(i, j int) bool {
    return factors[i].Cmp(factors[j]) < 0
})
var qs []*big.Int
for i, j := 0, 0; i < len(factors); i = j {
    q := new(big.Int).Set(factors[i])
    for j = i + 1; j < len(factors) ∧ factors[j].Cmp(factors[i]) ≡ 0; j++ {
        q.Mul(q, factors[j])
    }
    qs = append(qs, q)
}

```

이 코드는 85 절에서 쓰인다.

87. 조각이 작으면 사전을 쓰는 Shanks가, 크면 공간 걱정 없는 ρ 가 낫다. 경계는 2^{32} 로 잡았다—사전 항목 2^{16} 개쯤은 껌값이다.

〈 부분군마다 작은 이산 로그를 푼다 87〉 ≡

```

var dLogs []*big.Int
for _, q := range qs {
    t := new(big.Int).Div(N, q)
    gx, gy := c.ScalarMult(px, py, t)
    bx, by := c.ScalarMult(hx, hy, t)
    var k *big.Int
    if q.BitLen() ≤ 32 {
        k = c.shank(gx, gy, bx, by, q)
    } else {
        k = c.pollardRho(gx, gy, bx, by, q)
    }
    if k ≡  $\Lambda$  {
        return  $\Lambda$ 
    }
    dLogs = append(dLogs, k)
}

```

이 코드는 85 절에서 쓰인다.

88. 장난감 곡선($N = 19$)에서 Shanks와 ρ 가 숨긴 k 를 도로 찾는지 본다. Shanks의 답 $a + mb$ 는 N 을 넘을 수 있으니 $\text{mod } N$ 으로 견준다.

(`elliptic_test.go 3`) +=

```
func TestShankAndRho(t *testing.T) {
    c := toyCurve()
    k := big.NewInt(13)
    hx, hy := c.ScalarBaseMult(k)
    for name, dlp := range map[string]func(a, b, x, y *big.Int) *big.Int {
        "Shank": c.Shank, "PollardRho": c.PollardRho,
    } {
        got := dlp(c.Gx, c.Gy, hx, hy)
        if got ≡ Λ ∨ new(big.Int).Mod(got, c.N).Cmp(k) ≠ 0 {
            t.Fatalf("%s: k=13을 %v 로 잘못 찾았다", name, got)
        }
    }
}
```

89. Pohlig–Hellman은 위수가 매끄러운 제물이 필요하니 F_{1019} 위에서 직접 사냥한다: x 를 훑으며 곡선 위의 점을 찾고($1019 \equiv 3 \pmod{4}$ 라 제공근이 거듭제곱 한 방이다), 그 위수를 전수 덧셈으로 재서 합성수 위수인 점을 밑점으로 삼는다. 그런 다음 무작위 k 를 숨기고 도로 찾게 한다.

(`elliptic_test.go 3`) +=

```
func TestPohligHellman(t *testing.T) {
    const p = 1019
    c := &Curve{P: big.NewInt(p), A: big.NewInt(5), B: big.NewInt(7), BitSize: 10}
    f := NewFp(p)
    for x := uint64(1); x < p; x++ {
        r := f.add(f.mul(f.mul(x, x), x), f.add(f.mul(5, x), 7))
        y := f.pow(r, (p + 1)/4)
        if f.mul(y, y) ≠ r {
            continue
        }
        < 이 점의 위수를 재고, 합성수면 밑점으로 채택한다 90 >
    }
    if c.N ≡ Λ {
        t.Fatalf("합성수 위수의 점을 찾지 못했다")
    }
    k := new(big.Int).Mod(big.NewInt(int64(rng.Uint64())), c.N)
    hx, hy := c.ScalarBaseMult(k)
    got := c.PohligHellman(c.Gx, c.Gy, hx, hy)
    if got ≡ Λ ∨ got.Cmp(k) ≠ 0 {
        t.Fatalf("N=%v에서 k=%v를 %v로 잘못 찾았다", c.N, k, got)
    }
}
```

90. 위수 재기는 $P, 2P, 3P, \dots$ 를 $\mathcal{O}$ 가 나올 때까지 미련하게 더한다. p 가 천 남짓이라 하세 상한으로도 몇천 걸음이다.

〈 이 점의 위수를 재고, 합성수면 밑점으로 채택한다 90〉 $\equiv$

```

gx := new(big.Int).SetUint64(x)
gy := new(big.Int).SetUint64(y)
d := int64(1)
for tx, ty := new(big.Int).Set(gx), new(big.Int).Set(gy);  $\neg$ isInf(tx, ty); d++ {
    tx, ty = c.Add(tx, ty, gx, gy)
}
if d > 4  $\wedge$   $\neg$ big.NewInt(d).ProbablyPrime(20) {
    c.Gx, c.Gy, c.N = gx, gy, big.NewInt(d)
    break
}

```

이 코드는 89절에서 쓰인다.

91. ECDSA. 마지막 장은 앞의 모든 것을 조립해 실제로 돈이 오가는 물건을 만든다. 타원 곡선 디지털 서명 알고리즘(ECDSA)이다. 서명의 요구는 셋이다: 개인 키 d 를 가진 사람만 서명을 만들 수 있고, 공개 키 $Q = dG$ 만 아는 누구든 검증할 수 있고, 서명에서 d 가 새어 나가면 안 된다. 이산 로그가 어려운 한 셋째가 지켜진다는 것이 앞 장의 교훈이다.

먼저 열쇠부터. 개인 키는 $[1, N - 1]$ 의 무작위 정수, 공개 키는 그 배수다. 암호학적 난수원을 밖에서 주입받는다—시험에서는 *crypto/rand*를 쫓는다.

⟨ECDSA 91⟩ ≡

```
func (c *Curve) GenerateKey(rnd io.Reader)(priv, x, y *big.Int, err error) {
    nMinus1 := new(big.Int).Sub(c.N, big.NewInt(1))
    if priv, err = rand.Int(rnd, nMinus1); err != Λ {
        return
    }
    priv.Add(priv, big.NewInt(1))
    x, y = c.ScalarBaseMult(priv)
    return
}
```

이 코드는 92, 93, 94, 95절에도 정의되어 있다.

이 코드는 2절에서 쓰인다.

92. 서명 대상은 메시지의 해시다. FIPS 186-4의 지시대로 해시의 왼쪽 비트들을 위수의 비트 길이에 맞춰 자른다.

⟨ECDSA 91⟩ +=

```
func (c *Curve) hashToInt(hash []byte) *big.Int {
    orderBits := c.N.BitLen()
    orderBytes := (orderBits + 7) / 8
    if len(hash) > orderBytes {
        hash = hash[:orderBytes]
    }
    ret := new(big.Int).SetBytes(hash)
    if excess := len(hash) * 8 - orderBits; excess > 0 {
        ret.Rsh(ret, uint(excess))
    }
    return ret
}

func FermatInverse(k, n *big.Int) *big.Int {
    return new(big.Int).Exp(k, new(big.Int).Sub(n, big.NewInt(2)), n)
}
```

93. 서명. 일회용 비밀 k 를 뽑아 $R = kG$ 의 x 좌표를 r 로 삼고,

$$s = k^{-1}(z + rd) \bmod N$$

을 짝지으면 (r, s) 가 서명이다. r 나 s 가 0이면 다시 뽑는다.

여기서 잔소리를 늘어놓을 대목이 있다. k 는 서명마다 새로, 예측 불가능하게 뽑아야 한다. 같은 k 를 두 번 쓰면 두 서명에서 k 가, 이어서 개인 키 $d = (sk - z)r^{-1}$ 가 초등 대수로 풀려 나온다. 농담 같지만 2010년 소니 플레이스테이션 3가 정확히 이 사고를 쳤다—펌웨어 서명에 k 를 상수로 박아 둔 것이다. 해커 그룹 fail0verflow는 연말 발표회에서 칠판 두 줄로 소니의 마스터 키를 유도해 보였다. 백억 달러짜리 보안이 중학교 연립방정식에 무너진 순간이니, 아래 *GenerateKey* 호출 한 줄에는 그 수업료가 들어 있는 셈이다.

⟨ECDSA 91⟩ +=

```
func (c *Curve) Sign(priv *big.Int, hash []byte)(r, s *big.Int) {
    N := c.N
    z := c.hashToInt(hash)
    for {
        k, x, _, err := c.GenerateKey(rand.Reader)
        if err != Λ {
            continue
        }
        r = x.Mod(x, N)
        if r.Sign() == 0 {
            continue
        }
        s = new(big.Int).Mul(r, priv)
        s.Add(s, z)
        s.Mul(s, FermatInverse(k, N))
        s.Mod(s, N)
        if s.Sign() != 0 {
            return
        }
    }
}
```

94. 검증은 서명식을 뒤집은 것이다. $u_1 = zs^{-1}$, $u_2 = rs^{-1}$ 로 놓고 $u_1G + u_2Q$ 를 계산하면, 서명이 진짜일 때 이 점이 kG 와 같아지므로 그 좌표가 r 와 일치해야 한다. 대수 한 줄 계산: $u_1 + u_2d = (z + rd)s^{-1} = k \pmod{N}$.

⟨ECDSA 91⟩ +=

```
func (c *Curve) Verify(qx, qy *big.Int, hash []byte, r, s *big.Int) bool {
    N := c.N
    if r.Sign() ≤ 0 ∨ s.Sign() ≤ 0 ∨ r.Cmp(N) ≥ 0 ∨ s.Cmp(N) ≥ 0 {
        return false
    }
    sInv := FermatInverse(s, N)
    u1 := c.hashToInt(hash)
    u1.Mul(u1, sInv).Mod(u1, N)
    u2 := new(big.Int).Mul(r, sInv)
    u2.Mod(u2, N)
    x, y := c.CombinedMult(qx, qy, u1, u2)
    if isInf(x, y) {
        return false
    }
    return x.Mod(x, N).Cmp(r) == 0
}
```

95. 무대에 세울 곡선은 secp256k1이다. NIST 표준 곡선들(P-256 등)의 계수가 출처 불명의 상수를 품은 것과 달리 이 곡선은 $A = 0$, $B = 7$ —“소매에 아무것도 숨기지 않은” 담백한 선택이라, NSA가 표준 난수 생성기 DualEC.DRBG에 뒷문을 심었다는 사실이 드러난 뒤로 부쩍 사랑받았다. 사토시 나카모토가 비트코인에 채택 하면서 지금은 지구에서 가장 부지런히 일하는 곡선이 되었다 — 이 순간에도 초당 수천 번씩 서명하고 검증한다. $p = 2^{256} - 2^{32} - 977$ 이고 위수 N 은 소수다(Pohlig–Hellman 장의 교환 그대로).

⟨ECDSA 91⟩ +=

```
var S256 = &Curve{
    Name: "secp256k1",
    P: bigFromHex("fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f"),
    A: big.NewInt(0),
    B: big.NewInt(7),
    Gx: bigFromHex("79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798"),
    Gy: bigFromHex("483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8"),
    N: bigFromHex("fffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141"),
    H: big.NewInt(1),
    BitSize: 256,
}

func bigFromHex(s string) *big.Int {
    b, ok := new(big.Int).SetString(s, 16)
    if !ok {
        panic("elliptic: 잘못된 16진수 상수")
    }
    return b
}
```


96. `secp256k1`이 제대로 박혔는지(G 가 곡선 위에, $NG = \mathcal{O}$), 그리고 서명-검증 왕복이 도는지 본다. 위조 검사로는 다른 메시지의 해시를 들이민다.

`<elliptic_test.go 3> +≡`

```
func TestS256(t *testing.T) {
    c := S256
    if ¬c.IsOnCurve(c.Gx, c.Gy) {
        t.Fatal("G가 곡선 위에 없다")
    }
    if x, y := c.ScalarBaseMult(c.N); ¬isInf(x, y) {
        t.Fatal("NG! = 0")
    }
}

func TestECDSA(t *testing.T) {
    priv, qx, qy, err := S256.GenerateKey(rand.Reader)
    if err ≠ Λ {
        t.Fatal(err)
    }
    hash := sha256.Sum256([]byte("타원 곡선과 그 암호"))
    r, s := S256.Sign(priv, hash[:])
    if ¬S256.Verify(qx, qy, hash[:], r, s) {
        t.Fatal("참 서명이 검증에 떨어졌다")
    }
    forged := sha256.Sum256([]byte("위조된 메시지"))
    if S256.Verify(qx, qy, forged[:], r, s) {
        t.Fatal("위조 서명이 검증을 통과했다")
    }
}
```

97. 짐을 풀며. 짐을 풀 때가 되었다. 시계 산술에서 출발해 비트코인의 곡선에서 서명을 받아 들었으니, 여정치고는 제법 멀리 왔다.

돌아보면 같은 얼굴을 여러 고개에서 만났다. 현-접선 공식은 곡선장에서 수를 받아 점을 더하더니, Schoof 장에서는 다항식을 받아 사상을 더했다 — 옷만 갈아입은 같은 기하다. 중국인의 나머지 정리는 세 번 나왔다: NTT의 계수를 꿰맬 때, $t \bmod \ell$ 조각들을 이을 때, Pohlig–Hellman이 흠친 조각들을 맞추는 때. 큰 문제를 작은 조각으로 쪼개 각개격파하고 도로 꿰매는 이 한 수가 이 답사기의 숨은 주인공인 셈이다. 페르마의 작은 정리는 역원이 필요한 곳마다 굿은일을 도맡았고, Pollard의 ρ 는 이산 로그와 인수분해 양쪽에서 같은 곡조를 불렀다. 그리고 배운 것 하나—수학이 방패와 창을 가리지 않는다는 것. 점을 세던 프로베니우스와 위수를 쪼개던 Pohlig–Hellman은 적이 아니라, 좋은 곡선을 고르는 한 저울의 두 접시였다.

속도에 관한 교훈은 몸으로 얻었다. *math/big*으로 며칠 밤을 새우던 점 세기가 `uint64`와 NTT 위에서는 차 한 잔 식기 전에 끝난다. 알고리즘의 접근 기호가 같아도 상수는 이렇게 다르다—힙에 놓는 수와 레지스터를 달리는 수의 차이를, 표가 아니라 새벽으로 배웠다.

물론 지도에는 가 보지 못한 길이 더 많다. ψ_ℓ 을 작은 인수로 갈아 끼우는 Elkies와 Atkin의 지름길(SEA), 나눗셈을 아끼는 사영 좌표, 덧셈 공식에 예외가 없는 Edwards 곡선과 Curve25519, 두 곡선을 겹치는 페어링. 그리고 지평선 너머에는 Shor의 알고리즘이 있다—양자 컴퓨터가 서면 이산 로그는 다항 시간에 풀리고, 이 답사기의 암호 절반은 유적지가 된다. 그날이 와도 곡선 자체는 유적이 되지 않는다. 다음 세대 암호의 유력 후보 하나(아이소제니)가 다른 아닌 타원 곡선 사이의 길이니, 이 곡선과의 인연은 꽤 길게 갈 모양이다.

채트윈은 «송라인»에서 노래로 대륙의 길을 외는 사람들 이야기를 썼다. 조상들이 노래한 길을 따라 걸으면 사막에서도 길을 잃지 않는단다. 문학적 프로그램도 결국 그런 것이라고 믿는다—코드만 남기면 사막이지만, 사연과 함께 남기면 길이 된다. 이 답사기가 누군가에게 그런 노랫길이기.

$$y^2 = x^3 + Ax + B$$

끝으로, 여행 안내인 없이 이 길을 처음 낸 이들—Hasse, Schoof, Shanks, Pollard, Koblitz, Miller—에게 답사객의 예를 표한다.

98. 찾아보기.

- A*: [9](#), [35](#), [36](#), [40](#), [43](#), [46](#), [47](#), [48](#), [49](#), [51](#), [56](#), [57](#), [58](#),
[60](#), [63](#), [66](#), [68](#), [72](#), [73](#), [74](#), [89](#), [95](#).
a: [5](#), [6](#), [7](#), [8](#), [9](#), [12](#), [13](#), [14](#), [19](#), [20](#), [21](#), [28](#), [32](#), [33](#), [65](#),
[71](#), [72](#), [77](#), [78](#), [79](#), [88](#).
a1: [56](#), [57](#), [80](#), [81](#), [82](#).
a2: [48](#), [49](#), [56](#), [80](#), [82](#).
a3: [49](#), [56](#), [57](#).
ab: [49](#).
Abs: [41](#).
Add: [9](#), [12](#), [33](#), [36](#), [38](#), [40](#), [41](#), [42](#), [45](#), [56](#), [64](#), [65](#), [67](#),
[68](#), [69](#), [76](#), [77](#), [78](#), [79](#), [80](#), [81](#), [84](#), [90](#), [91](#), [93](#).
add: [5](#), [9](#), [12](#), [14](#), [16](#), [71](#), [72](#), [74](#), [89](#).
addConst: [14](#), [57](#).
addmod: [5](#), [6](#), [19](#), [23](#).
adif: [56](#).
ai: [16](#).
append: [11](#), [24](#), [67](#), [83](#), [86](#), [87](#).
As: [61](#).
B: [9](#), [35](#), [36](#), [43](#), [46](#), [47](#), [48](#), [49](#), [51](#), [60](#), [66](#), [68](#), [72](#),
[73](#), [74](#), [89](#), [95](#).
b: [5](#), [6](#), [8](#), [9](#), [12](#), [13](#), [21](#), [28](#), [32](#), [33](#), [71](#), [72](#), [78](#), [79](#),
[88](#), [95](#).
b1: [56](#), [57](#), [80](#), [81](#), [82](#).
b2: [49](#), [56](#), [80](#), [82](#).
b3: [56](#), [57](#).
baby: [77](#), [78](#).
base: [27](#).
big: [1](#), [2](#), [4](#), [9](#), [31](#), [35](#), [36](#), [37](#), [38](#), [39](#), [40](#), [41](#), [42](#), [43](#),
[44](#), [45](#), [46](#), [64](#), [65](#), [66](#), [67](#), [68](#), [69](#), [70](#), [71](#), [72](#), [73](#),
[74](#), [76](#), [77](#), [78](#), [79](#), [80](#), [82](#), [83](#), [84](#), [85](#), [86](#), [87](#), [88](#),
[89](#), [90](#), [91](#), [92](#), [93](#), [94](#), [95](#), [97](#).
bigFromHex: [95](#).
Bit: [41](#), [64](#), [84](#).
bit: [20](#).
BitLen: [41](#), [66](#), [87](#), [92](#).
bits: [6](#).
BitSize: [35](#), [43](#), [89](#), [95](#).
bj: [16](#).
bx: [87](#).
by: [87](#).
c: [10](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [22](#), [24](#), [25](#), [26](#), [29](#), [30](#),
[31](#), [36](#), [37](#), [38](#), [39](#), [40](#), [41](#), [42](#), [44](#), [45](#), [46](#), [47](#), [49](#),
[50](#), [51](#), [60](#), [65](#), [66](#), [67](#), [69](#), [72](#), [73](#), [74](#), [75](#), [76](#), [77](#),
[78](#), [79](#), [81](#), [82](#), [84](#), [85](#), [87](#), [88](#), [89](#), [90](#), [91](#), [92](#), [93](#),
[94](#), [96](#).
c0: [49](#).
cache: [47](#), [48](#), [49](#), [50](#).
cases: [31](#).
check: [9](#).
chord: [38](#), [39](#), [40](#).
clone: [11](#), [14](#), [24](#), [26](#).
Cmp: [36](#), [38](#), [44](#), [45](#), [64](#), [67](#), [69](#), [72](#), [73](#), [77](#), [78](#), [80](#),
[82](#), [83](#), [84](#), [86](#), [88](#), [89](#), [94](#).
coef: [25](#).
coeffs: [10](#).
CombinedMult: [42](#), [94](#).
copy: [12](#).
CRT: [65](#), [69](#), [85](#).
crypto: [3](#), [91](#).
Curve: [35](#), [36](#), [37](#), [38](#), [39](#), [40](#), [41](#), [42](#), [43](#), [46](#), [66](#),
[72](#), [73](#), [74](#), [76](#), [79](#), [85](#), [89](#), [91](#), [92](#), [93](#), [94](#), [95](#).
cycle: [84](#).
d: [26](#), [38](#), [40](#), [47](#), [83](#), [90](#).
Deg: [10](#), [24](#), [26](#), [29](#), [33](#), [51](#).
den: [50](#), [57](#), [82](#).
disc: [72](#).
Div: [65](#), [83](#), [87](#).
Div64: [6](#).
DivMod: [24](#), [26](#), [29](#), [33](#), [50](#).
divPoly: [46](#), [47](#), [50](#), [51](#), [60](#).
dLogs: [85](#), [87](#).
dlp: [88](#).
Done: [68](#).
Double: [38](#), [40](#), [41](#), [79](#).
dp: [46](#), [47](#), [50](#).
e: [7](#), [8](#), [27](#), [55](#).
ell: [60](#), [63](#).
elliptic: [2](#), [3](#).
ells: [67](#), [68](#), [69](#).
endo: [54](#), [56](#), [57](#), [58](#), [63](#).
endoAdd: [56](#), [58](#), [63](#).
endoDouble: [56](#), [57](#), [58](#).
endoEq: [54](#), [63](#).
endoNeg: [54](#), [63](#).
endoScalarMul: [58](#), [63](#).
equal: [11](#), [32](#), [33](#), [54](#), [56](#), [59](#).
err: [58](#), [60](#), [61](#), [63](#), [68](#), [72](#), [73](#), [74](#), [91](#), [93](#), [96](#).
errNoCharPoly: [60](#), [61](#), [63](#).
Error: [55](#).
errors: [60](#), [61](#), [66](#).
errs: [68](#).
excess: [92](#).
Exp: [92](#).

- f*: 5, 8, 9, 10, 11, 12, 13, 14, 15, 16, 23, 24, 25, 26, 27, 29, 30, 32, 33, 46, 47, 48, 49, 51, 53, 59, 60, 63, 66, 68, 71, 72, 74, 89.
fa: 21.
factor: 55, 61, 84.
factorize: 83, 86.
factors: 83, 86.
Fatal: 33, 44, 45, 72, 73, 74, 89, 96.
Fatalf: 9, 31, 32, 44, 51, 72, 73, 74, 88, 89.
fb: 21.
FermatInverse: 92, 93, 94.
forged: 96.
Fp: 4, 5, 6, 8, 10, 30, 46, 53, 60.
fpCurve: 46, 47, 51, 60.
FpPoly: 10, 11, 12, 13, 14, 15, 16, 22, 24, 26, 27, 28, 29, 30, 46, 47, 50, 53, 54, 55, 56, 57, 58.
fromInt: 5, 48, 49.
fsq: 67.
fx: 56, 57, 58, 60, 62, 63.
g: 19, 21, 31.
GCD: 28, 59, 61, 65, 84.
GenerateKey: 91, 93, 96.
got: 9, 51, 72, 73, 88, 89.
Gx: 35, 42, 43, 44, 88, 89, 90, 95, 96.
gx: 87, 90.
Gy: 35, 42, 43, 44, 88, 89, 90, 95, 96.
gy: 87, 90.
H: 35, 43, 95.
h: 29, 33, 53, 56, 57, 59, 61, 62.
half: 19.
hash: 92, 93, 94, 96.
hashToInt: 92, 93, 94.
hi: 6.
hx: 76, 78, 79, 81, 82, 85, 87, 88, 89.
hy: 76, 78, 79, 81, 82, 85, 87, 88, 89.
i: 10, 11, 12, 13, 14, 16, 19, 20, 21, 23, 25, 30, 41, 58, 65, 68, 86.
id: 63.
Int: 2, 4, 9, 31, 35, 36, 37, 38, 39, 40, 41, 42, 45, 46, 64, 65, 66, 67, 68, 69, 70, 71, 72, 74, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95.
Int64: 65, 68, 79, 80.
inv: 8, 9, 24, 26, 29, 33, 56, 57.
inv12: 23.
inv123: 23.
invert: 19.
io: 91.
irreducible: 59, 60.
Is: 61.
isInf: 37, 38, 44, 74, 90, 94, 96.
IsInt64: 80.
IsOnCurve: 36, 44, 96.
isQR: 71.
isZero: 10, 28, 29, 33.
j: 10, 16, 19, 20, 80, 84, 86.
k: 41, 42, 82, 87, 88, 89, 92, 93.
kk: 41.
l: 67.
len: 10, 11, 12, 13, 14, 15, 16, 19, 24, 25, 68, 86, 92.
length: 19.
li: 80.
limit: 80.
lo: 6.
Lsh: 40, 79.
M: 67, 69.
m: 6, 7, 19, 21, 27, 31, 38, 39, 40, 42, 50, 56, 57, 76, 77, 78, 83.
m12: 23.
m12p: 23.
m1p: 23.
m2: 56.
make: 10, 12, 13, 14, 16, 21, 23, 24, 30, 47, 68, 71, 77.
math: 1, 3, 4, 6, 9, 66, 97.
max: 13.
Mod: 9, 26, 27, 28, 29, 33, 36, 37, 38, 39, 40, 53, 65, 70, 79, 82, 84, 88, 89, 93, 94.
ModInverse: 29, 33, 38, 40, 55, 56, 57, 82.
Monic: 26, 28, 60.
monic: 24, 25.
mrnd: 3, 9.
Mul: 9, 15, 18, 27, 29, 32, 33, 36, 38, 39, 40, 49, 50, 56, 57, 65, 67, 78, 80, 82, 84, 86, 93, 94.
mul: 8, 9, 14, 16, 25, 48, 49, 71, 72, 74, 89.
Mul64: 6.
mulmod: 6, 7, 8, 19, 21, 23.
mulNTT: 15, 22.
MulScalar: 14, 26, 29, 47, 50, 57.
mulSchool: 15, 16, 32.
mulThreshold: 15.
N: 35, 43, 44, 75, 76, 79, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96.
n: 11, 13, 15, 19, 20, 21, 22, 25, 30, 42, 47, 50, 51, 58, 64, 65, 71, 74, 76, 79, 80, 81, 82, 83, 84, 92.
naiveCount: 71, 72, 73.
Name: 35, 43, 95.
name: 88.

- Neg*: 14, 37, 44, 54, 78.
neg: 5, 14, 48, 49.
New: 9, 60, 66.
new: 9, 31, 36, 37, 38, 39, 40, 41, 64, 67, 69, 70, 71, 72, 74, 76, 77, 78, 79, 80, 82, 83, 84, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95.
newEnd: 54, 56, 57, 58, 62, 63.
NewFp: 4, 9, 32, 33, 51, 66, 71, 72, 74, 89.
NewInt: 40, 43, 44, 45, 64, 65, 67, 68, 69, 73, 74, 76, 77, 78, 79, 80, 83, 84, 88, 89, 90, 91, 92, 95.
NewPCG: 9.
newR: 29.
newT: 29.
NextPrime: 64, 67.
ninv: 19.
nMinus1: 91.
ntt: 19, 21.
ntt1: 18, 22, 23, 31.
ntt1g: 18, 22, 31.
ntt2: 18, 22, 23, 31.
ntt2g: 18, 22, 31.
ntt3: 18, 22, 23, 31.
ntt3g: 18, 22, 31.
nttConv: 21, 22.
num: 57.
nx: 44.
ny: 37, 44.
ok: 29, 33, 47, 56, 57, 78, 95.
one: 84.
op: 9.
orderBits: 92.
orderBytes: 92.
P: 9, 35, 36, 37, 38, 39, 40, 43, 63, 66, 67, 69, 70, 72, 73, 74, 89, 95.
p: 4, 5, 8, 10, 11, 12, 13, 14, 15, 16, 22, 23, 24, 26, 27, 28, 29, 30, 32, 53, 60, 64, 66, 70, 71, 72, 73, 74, 89.
p1m: 50.
p2m: 50.
p61: 9, 32.
panic: 4, 95.
pe: 54, 56, 57, 58.
pi: 62, 63.
pi2: 62, 63.
pm: 23, 50.
pm1: 50.
pm2: 50.
PohligHellman: 85, 89.
PollardRho: 79, 88.
pollardRho: 79, 87.
Poly: 10, 24, 27, 29, 33, 46, 47, 48, 49, 59, 60, 63.
poly: 46, 47, 49, 50, 60.
pow: 8, 74, 89.
PowMod: 27, 59, 62.
powmod: 7, 8, 19, 23, 31.
priv: 91, 93, 96.
ProbablyPrime: 31, 64, 83, 90.
prod: 65.
pt: 45.
ptKey: 76, 77, 78.
px: 41, 45, 74, 76, 77, 78, 79, 81, 82, 85, 87.
py: 41, 45, 74, 76, 77, 78, 79, 81, 82, 85, 87.
Q: 63.
q: 11, 12, 13, 15, 16, 22, 24, 25, 26, 28, 31, 33, 59, 60, 62, 65, 86, 87.
qd: 24, 25.
qe: 54, 56.
qHalf: 60, 62.
qi: 25.
qInv: 24, 25.
qModEll: 60, 63.
qr: 53, 54, 56, 57, 58, 59, 60, 61, 62, 63.
qring: 53, 54, 59, 60.
qs: 31, 85, 86, 87.
quo: 24, 29.
quoC: 24, 25.
qx: 42, 45, 94, 96.
qy: 42, 45, 94, 96.
r: 5, 6, 7, 12, 13, 14, 16, 22, 23, 26, 27, 29, 33, 36, 71, 74, 89, 93, 94, 96.
r1: 22, 23.
r2: 22, 23.
r3: 22, 23.
rand: 3, 81, 91, 93, 96.
randPoly: 30, 32, 33.
rd: 25.
re: 58.
Reader: 81, 91, 93, 96.
reduce: 5, 10, 14, 53, 54, 56, 57.
reduceBig: 66, 70.
rem: 24.
remC: 24, 25.
res: 69.
ret: 92.
rhoFactor: 83, 84.
rhs: 36.
rlen: 22, 23.

- rnd*: [91](#).
- rng*: [9](#), [30](#), [33](#), [45](#), [72](#), [89](#).
- Rsh*: [69](#), [92](#).
- rx*: [45](#), [77](#), [78](#).
- ry*: [45](#), [77](#), [78](#).
- S*: [63](#).
- s*: [6](#), [38](#), [65](#), [93](#), [94](#), [95](#), [96](#).
- S256*: [95](#), [96](#).
- ScalarBaseMult*: [42](#), [44](#), [45](#), [88](#), [89](#), [91](#), [96](#).
- ScalarMult*: [41](#), [42](#), [74](#), [78](#), [81](#), [82](#), [87](#).
- Schoof*: [66](#), [72](#), [73](#), [74](#).
- Set*: [37](#), [38](#), [64](#), [67](#), [77](#), [78](#), [83](#), [84](#), [86](#), [90](#).
- SetBytes*: [92](#).
- SetString*: [95](#).
- SetUint64*: [9](#), [31](#), [70](#), [71](#), [72](#), [74](#), [90](#).
- sha256*: [96](#).
- Shank*: [76](#), [88](#).
- shank*: [76](#), [87](#).
- Sign*: [36](#), [37](#), [38](#), [40](#), [66](#), [82](#), [93](#), [94](#), [96](#).
- sInv*: [94](#).
- Slice*: [86](#).
- sort*: [86](#).
- split*: [83](#).
- Sqrt*: [67](#), [76](#), [80](#).
- started*: [58](#).
- step*: [79](#), [80](#).
- Sub*: [9](#), [13](#), [29](#), [38](#), [39](#), [50](#), [56](#), [57](#), [59](#), [69](#), [82](#), [84](#), [91](#), [92](#).
- sub*: [5](#), [9](#), [13](#), [25](#), [49](#).
- submod*: [5](#), [6](#), [19](#), [23](#).
- Sum256*: [96](#).
- sx*: [78](#).
- sy*: [78](#).
- sync*: [68](#).
- T*: [9](#), [31](#), [32](#), [33](#), [44](#), [51](#), [72](#), [73](#), [74](#), [88](#), [89](#), [96](#).
- t*: [9](#), [29](#), [31](#), [32](#), [33](#), [44](#), [45](#), [51](#), [63](#), [68](#), [69](#), [72](#), [73](#), [74](#), [87](#), [88](#), [89](#), [96](#).
- t1*: [50](#).
- t2*: [23](#), [50](#).
- t3*: [23](#).
- td*: [25](#).
- TestCurveGroup*: [44](#).
- TestDivPolyDeg*: [51](#).
- TestECDSA*: [96](#).
- TestFp*: [9](#).
- testing*: [9](#), [31](#), [32](#), [33](#), [44](#), [51](#), [72](#), [73](#), [74](#), [88](#), [89](#), [96](#).
- TestNTTModuli*: [31](#).
- TestPohligHellman*: [89](#).
- TestPolyDiv*: [33](#).
- TestPolyInverse*: [33](#).
- TestPolyMul*: [32](#).
- TestS256*: [96](#).
- TestSchoofMedium*: [73](#).
- TestSchoofOrder*: [74](#).
- TestSchoofSmall*: [72](#).
- TestShankAndRho*: [88](#).
- Text*: [76](#).
- toyCurve*: [43](#), [44](#), [88](#).
- traceMod*: [60](#), [68](#).
- traces*: [68](#), [69](#).
- trim*: [10](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [24](#).
- tx*: [82](#), [90](#).
- ty*: [82](#), [90](#).
- u*: [19](#).
- u1*: [94](#).
- u2*: [94](#).
- Uint64*: [9](#), [30](#), [33](#), [45](#), [66](#), [70](#), [72](#), [89](#).
- v*: [5](#), [10](#), [14](#), [19](#), [21](#), [23](#), [70](#).
- v2*: [3](#).
- Verify*: [94](#), [96](#).
- vx*: [81](#).
- vy*: [81](#).
- w*: [19](#).
- Wait*: [68](#).
- WaitGroup**: [68](#).
- want*: [9](#), [51](#), [72](#), [73](#).
- wg*: [68](#).
- wn*: [19](#).
- wx*: [81](#).
- wy*: [81](#).
- X*: [74](#).
- x*: [36](#), [37](#), [40](#), [41](#), [44](#), [54](#), [56](#), [57](#), [58](#), [59](#), [60](#), [62](#), [63](#), [65](#), [71](#), [74](#), [76](#), [79](#), [84](#), [88](#), [89](#), [90](#), [91](#), [93](#), [94](#), [96](#).
- x1*: [38](#), [39](#), [42](#), [45](#), [80](#), [81](#).
- x12*: [23](#).
- x2*: [38](#), [39](#), [42](#), [45](#), [80](#).
- x3*: [39](#).
- xq*: [59](#), [62](#).
- xs*: [84](#).
- Y*: [74](#).
- y*: [36](#), [37](#), [40](#), [41](#), [44](#), [54](#), [56](#), [57](#), [58](#), [63](#), [74](#), [76](#), [79](#), [88](#), [89](#), [90](#), [91](#), [94](#), [96](#).
- y1*: [38](#), [39](#), [42](#), [45](#), [80](#), [81](#).
- y2*: [36](#), [38](#), [42](#), [45](#), [80](#).
- y3*: [39](#).
- yq*: [62](#).

z : [65](#), [93](#).

zd : [61](#).

zeroDivError: [55](#), [56](#), [57](#), [61](#).

- 〈 ψ_3 를 만들어 돌려준다 48〉 47절에서 쓰임
- 〈 ψ_4 를 만들어 돌려준다 49〉 47절에서 쓰임
- 〈CRT로 t 를 복원하고 $\#E = p + 1 - t$ 를 낸다 69〉 66절에서 쓰임
- 〈ECDSA 91〉 2절에서 쓰임
- 〈`elliptic_test.go` 3〉
- 〈Garner의 방법으로 세 나머지에서 계수를 복원한다 23〉 22절에서 쓰임
- 〈Schoof 알고리즘 53〉 2절에서 쓰임
- 〈거인걸음: $H - b(mP)$ 가 사전에 있는지 훑아 본다 78〉 76절에서 쓰임
- 〈곰이 하세 구간을 덮을 때까지 소수 ℓ 을 모은다 67〉 66절에서 쓰임
- 〈나눗셈 다항식 46〉 2절에서 쓰임
- 〈다항식 10〉 2절에서 쓰임
- 〈몫의 자리를 하나씩 정하며 나머지를 깎는다 25〉 24절에서 쓰임
- 〈무작위 $a_1P + b_1H$ 에서 출발한다 81〉 80절에서 쓰임
- 〈무작위 배수들로 교환법칙과 결합법칙을 확인한다 45〉 44절에서 쓰임
- 〈무작위 출발점에서 토끼와 거북을 달리게 한다 80〉 79절에서 쓰임
- 〈부분군마다 작은 이산 로그를 푼다 87〉 85절에서 쓰임
- 〈소수 ℓ 마다 고루틴 하나가 $t \bmod \ell$ 을 구한다 68〉 66절에서 쓰임
- 〈아기걸음: aP 를 사전에 담는다 77〉 76절에서 쓰임
- 〈오류를 살핀다: 인수를 주웠으면 법을 좁히고, 가망 없으면 접는다 61〉 60절에서 쓰임
- 〈위수를 소수 거듭제곱들로 쪼갬다 86〉 85절에서 쓰임
- 〈유한체 4〉 2절에서 쓰임
- 〈이 점의 위수를 재고, 합성수면 밑점으로 채택한다 90〉 89절에서 쓰임
- 〈이산 로그 문제 76〉 2절에서 쓰임
- 〈점화식으로 ψ_n 을 만들어 돌려준다 50〉 47절에서 쓰임
- 〈첨자를 비트 뒤집기 순서로 재배열한다 20〉 19절에서 쓰임
- 〈충돌 장부에서 k 를 풀어 본다; 성공이면 답한다 82〉 80절에서 쓰임
- 〈타원 곡선 35〉 2절에서 쓰임
- 〈특성 방정식에 $\bar{t}$ 후보를 대 본다 63〉 60절에서 쓰임
- 〈프로베니우스 π 와 π^2 을 만든다 62〉 60절에서 쓰임

나의 타원곡선 답사기

	절	쪽
여행 배낭을 꾸리며	1	1
유한체 F_p	4	3
다항식	10	7
수론적 변환	17	11
다항식 나눗셈과 그 친구들	24	15
타원 곡선	34	21
나눗셈 다항식	46	28
Schoof 알고리즘	52	32
이산 로그 문제	75	45
ECDSA	91	54
짐을 풀며	97	58
찾아보기	98	59